

Skills Development Program Scheduling

Helmut Simonis

Constraint Based Scheduling (One Day Course)



Licence



This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only. If others modify or adapt the material, they must license the modified material under identical terms.



Acknowledgments



This publication was developed as part of the ENTIRE EDIH project, which received funding from Enterprise Ireland and the European Commission.

Part of this work is based on research conducted with the financial support of Science Foundation Ireland under Grant number 12/RC/2289-P2 at Insight the SFI Research Centre for Data Analytics at UCC, which is co-funded under the European Regional Development Fund.

Part of this work is based on research conducted within the ASSISTANT European project, under the framework program Horizon 2020, ICT-38-2020, Artificial intelligence for manufacturing, grant agreement number 101000165.

Note



- This is a document which combines all materials from the Scheduling course
- Files are also available individually in separate directories

Insight is one of the largest data research and innovation centres in Europe...



4 Co-Lead Universities 9 partner institutions	Built on 20 years of research in Data Analytics and AI
450+ Academics, Postdocs, PhDs, RAs	3400+ Scientific conference and journal papers
175+ Funded collaborations with industry partners	350+ Research Awards
16 Spin out companies 72 license agreements	135+ H2020 consortia, 500+ collaborations, 40+ countries
1,137+ school visits, 28,000 students	276 PhDs graduated

Rinn Artificial Intelligence: Research & Innovation in Data Science and AI



Rinn Artificial Intelligence is designed to serve as a national hub for research and innovation in data science and AI. The programme has two overarching goals: (i) to advance foundational cutting-edge research in Data Science and AI, and (ii) to pursue open, international, and inter- and transdisciplinary research aimed at studying, designing, and implementing strategies to address societal challenges.



Background



- Mathematics @ TH Darmstadt
- 1986-1990 ECRC GmbH, Munich
- 1990-2000, Technical Director, Cosytec SA, Orsay
- 2000-2005, Imperial College London, Parc Technologies Ltd
- 2013-2014, President, Association for Constraint Programming
- Best Application Paper Awards, CP 2009, CP 2013
- Program Chair, CP 2020, CPAIOR 2014
- Distinguished Service Award, ACP
- Editor in Chief, Constraints, 2026



Course Structure



Time	Topic
09:00-09:30	Registration & Welcome
09:30-10:30	Introduction & Motivation
10:30-11:00	Coffee
11:00-12:30	Scheduling Concepts
12:30-13:30	Lunch
13:30-15:00	Case Studies
15:00-15:30	Coffee
15:30-16:30	Hands-on Experience

Part I

Introduction

ENTIRE EDIH

Scheduling (One Day Course)

Slide 9

Key Points



- Introducing a running example
- AI is more than LLM
- Stochastic vs. deductive AI methods
- Constraint Based Scheduling and its alternatives
- Key advantages
 - Compositional
 - Reusable
 - Explainable
- Course structure

Developing a Generic Scheduling Tool



- No programming, configured by JSON input data
- Compositional use of different constraint types
- Different commercial or open-source back-end solvers
- Developed in Java
- Interactive JavaFX front-end
- Can be used as back-end scheduling tool/server
- Instance generator included
- Readers for multiple benchmark types included
- Release planned early 2025
- Preview during the course, hands-on experience this afternoon

Introducing a Simple Scheduling Problem



- Will be used throughout the program
- Generated by instance generator
- 50 orders for different products, release and due dates
- 4 stages, always performed in the same sequence
- Two identical machines available for each stage
- Cumulative manpower constraint
- Complete description as JSON document

Excerpt of JSON Description



```
1  "order": [  
2    {  
3      "product": "Prod0",  
4      "process": "Process 0",  
5      "due": 5449,  
6      "releaseDate": "1/10/2024 00:00",  
7      "release": 0,  
8      "qty": 7,  
9      "dueDate": "19/10/2024 22:05",  
10     "name": "Order0",  
11     "earlinessWeight": 1,  
12     "latenessWeight": 1  
13   },
```

Orders Loaded

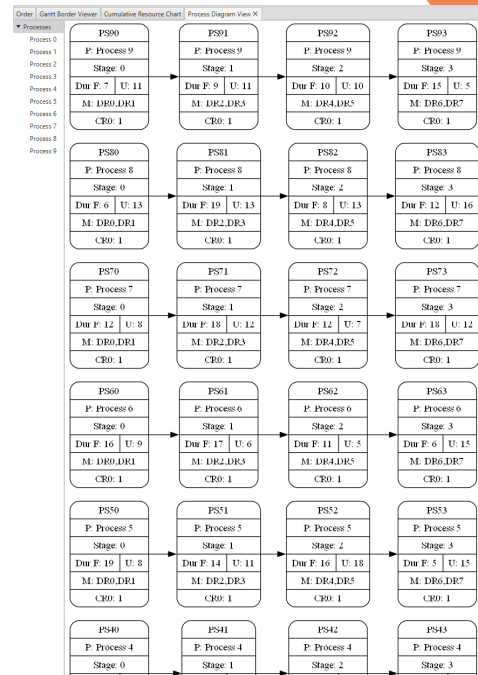


Order X										
Name	Nr	Product	Process	Qty	Due	DueDate	Release	ReleaseDate	LatenessWeight	EarlinessWeight
Order0	0	Prod0	Process 0	7	5,449	19/10/2024 22:05	0	1/10/2024 00:00	1.0	1.0
Order1	1	Prod1	Process 1	6	2,134	8/10/2024 09:50	0	1/10/2024 00:00	1.0	1.0
Order2	2	Prod1	Process 1	7	1,266	5/10/2024 09:30	0	1/10/2024 00:00	1.0	1.0
Order3	3	Prod1	Process 1	1	1,976	7/10/2024 20:40	0	1/10/2024 00:00	1.0	1.0
Order4	4	Prod9	Process 9	5	2,866	10/10/2024 22:50	0	1/10/2024 00:00	1.0	1.0
Order5	5	Prod9	Process 9	3	3,339	12/10/2024 14:15	0	1/10/2024 00:00	1.0	1.0
Order6	6	Prod4	Process 4	9	1,676	6/10/2024 19:40	0	1/10/2024 00:00	1.0	1.0
Order7	7	Prod5	Process 5	4	5,471	19/10/2024 23:55	0	1/10/2024 00:00	1.0	1.0
Order8	8	Prod8	Process 8	1	1,966	7/10/2024 19:50	0	1/10/2024 00:00	1.0	1.0
Order9	9	Prod8	Process 8	1	4,279	15/10/2024 20:35	0	1/10/2024 00:00	1.0	1.0
Order10	10	Prod9	Process 9	6	5,733	20/10/2024 21:45	0	1/10/2024 00:00	1.0	1.0
Order11	11	Prod4	Process 4	4	3,088	11/10/2024 17:20	0	1/10/2024 00:00	1.0	1.0
Order12	12	Prod8	Process 8	9	2,569	9/10/2024 22:05	0	1/10/2024 00:00	1.0	1.0
Order13	13	Prod7	Process 7	4	2,331	9/10/2024 02:15	0	1/10/2024 00:00	1.0	1.0
Order14	14	Prod4	Process 4	9	3,290	12/10/2024 10:10	0	1/10/2024 00:00	1.0	1.0
Order15	15	Prod3	Process 3	6	1,968	7/10/2024 20:00	0	1/10/2024 00:00	1.0	1.0
Order16	16	Prod4	Process 4	8	1,579	6/10/2024 11:35	0	1/10/2024 00:00	1.0	1.0
Order17	17	Prod1	Process 1	3	4,263	15/10/2024 19:15	0	1/10/2024 00:00	1.0	1.0
Order18	18	Prod5	Process 5	9	4,491	16/10/2024 14:15	0	1/10/2024 00:00	1.0	1.0
Order19	19	Prod3	Process 3	4	613	3/10/2024 03:05	0	1/10/2024 00:00	1.0	1.0
Order20	20	Prod6	Process 6	2	5,034	18/10/2024 11:30	0	1/10/2024 00:00	1.0	1.0
Order21	21	Prod7	Process 7	4	1,797	7/10/2024 05:45	0	1/10/2024 00:00	1.0	1.0
Order22	22	Prod8	Process 8	7	4,286	15/10/2024 21:10	0	1/10/2024 00:00	1.0	1.0
Order23	23	Prod9	Process 9	8	1,970	7/10/2024 20:10	0	1/10/2024 00:00	1.0	1.0
Order24	24	Prod3	Process 3	4	1,286	5/10/2024 11:10	0	1/10/2024 00:00	1.0	1.0
Order25	25	Prod6	Process 6	6	4,170	15/10/2024 11:30	0	1/10/2024 00:00	1.0	1.0
Order26	26	Prod8	Process 8	4	5,481	20/10/2024 00:45	0	1/10/2024 00:00	1.0	1.0
Order27	27	Prod1	Process 1	4	3,255	12/10/2024 07:15	0	1/10/2024 00:00	1.0	1.0
Order28	28	Prod3	Process 3	7	1,021	4/10/2024 13:05	0	1/10/2024 00:00	1.0	1.0
Order29	29	Prod5	Process 5	4	5,315	19/10/2024 10:55	0	1/10/2024 00:00	1.0	1.0
Order30	30	Prod9	Process 9	7	5,075	18/10/2024 14:55	0	1/10/2024 00:00	1.0	1.0
Order31	31	Prod1	Process 1	6	3,089	11/10/2024 17:25	0	1/10/2024 00:00	1.0	1.0
Order32	32	Prod0	Process 0	8	3,324	12/10/2024 13:00	0	1/10/2024 00:00	1.0	1.0
Order33	33	Prod7	Process 7	9	607	3/10/2024 02:35	0	1/10/2024 00:00	1.0	1.0
Order34	34	Prod9	Process 9	1	2,914	11/10/2024 02:50	0	1/10/2024 00:00	1.0	1.0

Process Diagram



- Processes describe how products are made
- Multiple process steps
- Not always in a straight sequence
- Duration formula based on quantity made
- Temporal constraints between steps
- Possible machines to run on
- Resource requirements (manpower, electricity,...)

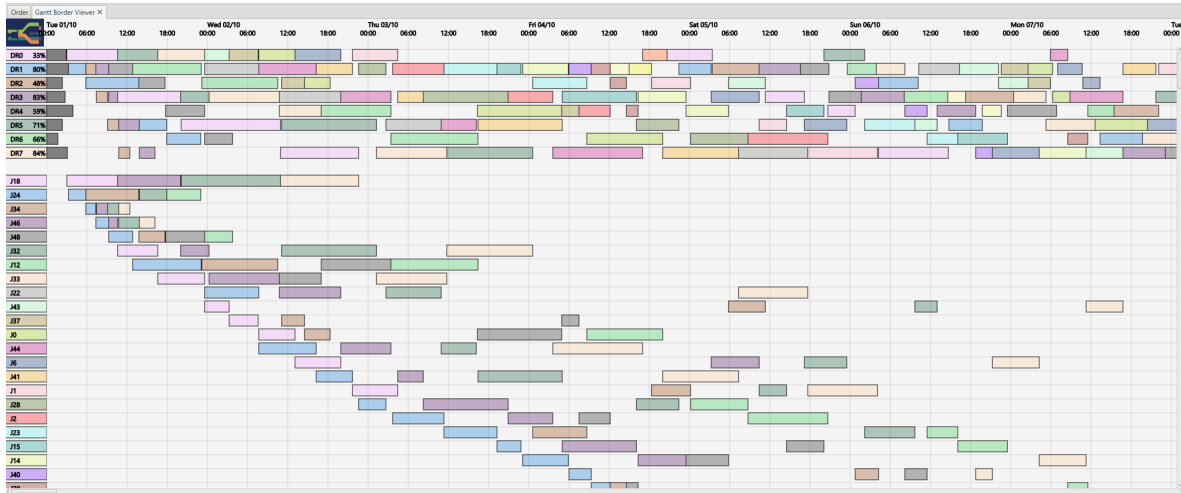


Selecting Solver Options



- Which constraints to enforce
 - Here: do not enforce due dates
- Additional constraints to try
- Why solver to run
 - Here: Use open-source CPSat solver
- Which objective to use
 - Here: Makespan, overall project end
- What resources to use
 - Allow 30 seconds
 - Use 8 parallel threads

Schedule - Initial Gantt Chart

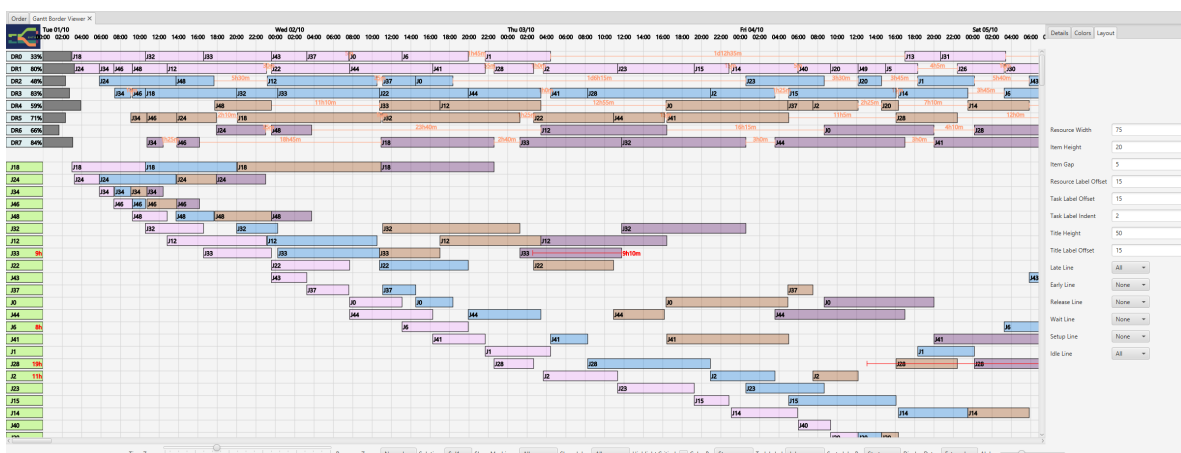


ENTIRE EDIH

Scheduling (One Day Course)

Slide 18

Adapted Gantt Chart

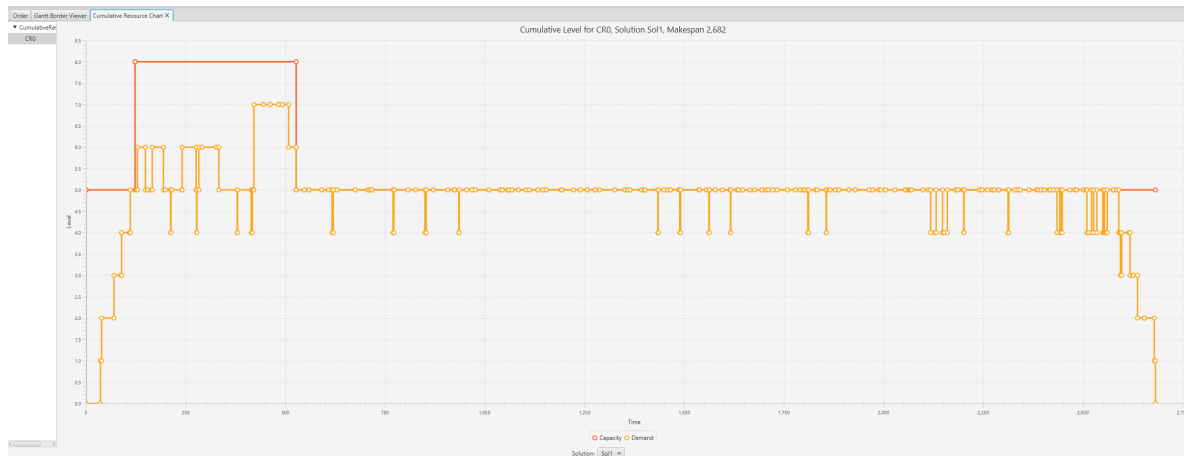


ENTIRE EDIH

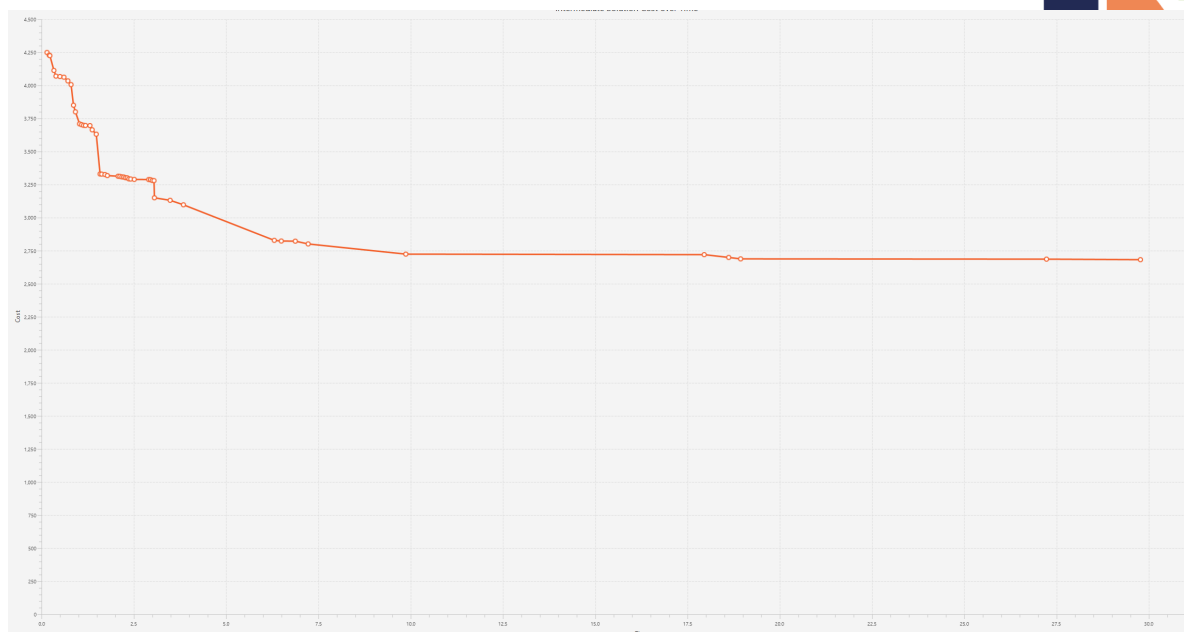
Scheduling (One Day Course)

Slide 19

Cumulative Resource Chart



Intermediate Solutions Found



- Ongoing search for improved solutions
- Depends on time and resources, solver used

What is Artificial Intelligence?



Artificial intelligence, or AI, is the field that studies the synthesis and analysis of computational agents that act intelligently.

David Poole, Alan Mackworth. Artificial Intelligence, Cambridge University Press, 3rd Edition, 2023.

- This definition leaves a lot of questions.

The Great Divide

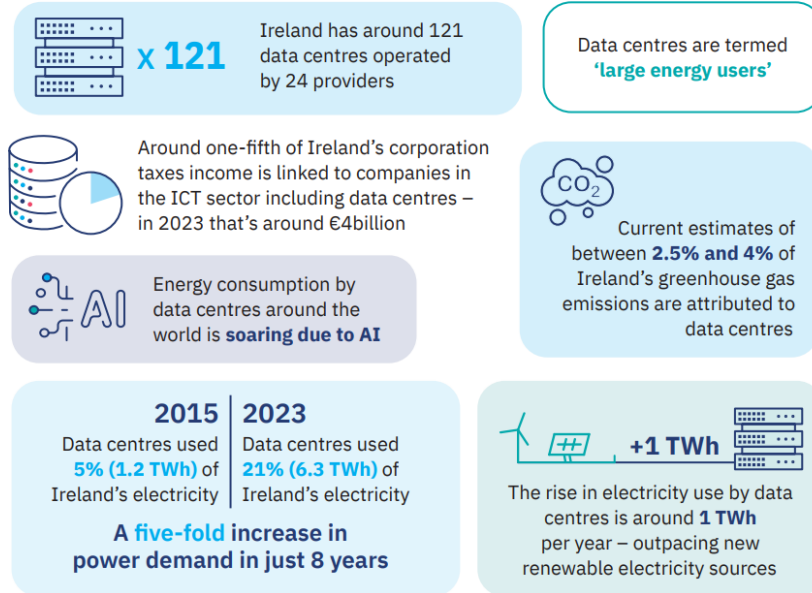


- Two fundamentally different approaches to AI
 - Reasoning based
 - Stochastic
- Currently, the stochastic methods get all the attention
- But they have their problems
 - Impossible to understand what is happening inside
 - Hallucinations, making up convincing false statements
 - Enormous resource requirements
 - Electricity, space, water
 - Privacy/IP of training data
 - Really limited to a few multi-nationals
 - Increasingly: cost to end-user

Datacentres in Ireland [Oireachtas 2024]



Data centres at a glance



https://data.oireachtas.ie/ie/oireachtas/libraryResearch/2025/2025-03-20_the-future-of-data-centres-in-ireland_en.pdf

ENTIRE EDIH

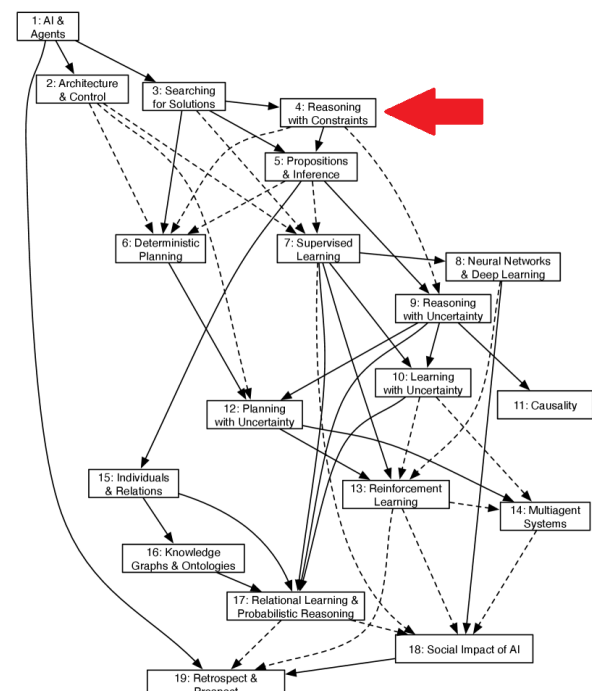
Scheduling (One Day Course)

Slide 25

Topics in AI



- Chapter Structure of AI Book
- Shows importance of deductive/search based approaches



ENTIRE EDIH

Scheduling (One Day Course)

Slide 26

What is Constraint Programming?



Constraint programming technology is used to find solutions to scheduling and combinatorial optimization problems. It is based primarily on computer science fundamentals, such as logic programming and graph theory, in contrast to mathematical programming, which is based on numerical linear algebra.

Constraint programming is invaluable when dealing with the complexity of many real-world sequencing and scheduling problems.

IBM (<https://ibmdecisionoptimization.github.io/docplex-doc/cp.html>)

What is Scheduling?



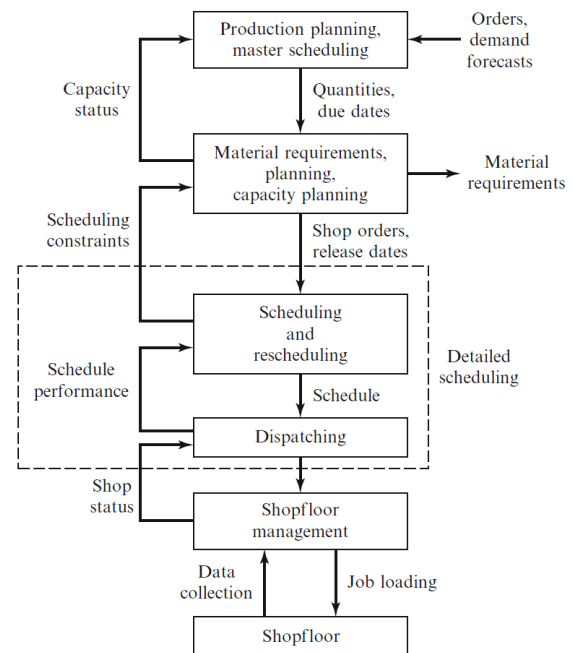
Scheduling is a decision-making process that is used on a regular basis in many manufacturing and services industries. It deals with the allocation of resources to tasks over given time periods and its goal is to optimize one or more objectives.

Michael Pinedo. Scheduling. Springer, 5th edition, 2016.

Information Flow Diagram in a Manufacturing System



- According to Pinedo, page 5.
- We focus on what is shown as *detailed scheduling*



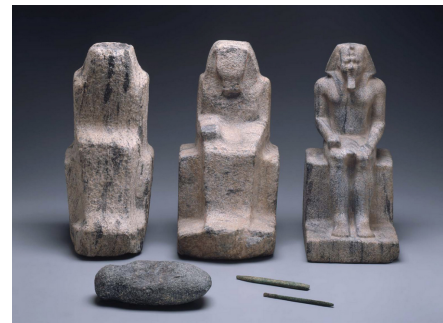
Constraint Programming - in a nutshell



- Declarative description of problems with
 - *Variables* which range over (finite) sets of values
 - *Constraints* over subsets of variables which restrict possible value combinations
 - A *solution* is a value assignment which satisfies all constraints
- Constraint propagation/reasoning
 - Removing inconsistent values for variables
 - Detect failure if constraint can not be satisfied
 - Interaction of constraints via shared variables
 - Incomplete
- Search
 - User controlled assignment of values to variables
 - Each step triggers constraint propagation
- Different domains require/allow different methods

- Declarative Programming
 - Concentrate on what you want
 - Not how to get there
 - Program $\neq$ Algorithm
 - Program = Model
- Applied to Combinatorial Problems
 - No complete polynomial algorithms known (exist?)
 - CP less ad-hoc than heuristics
 - Models can evolve

A Subtractive Process



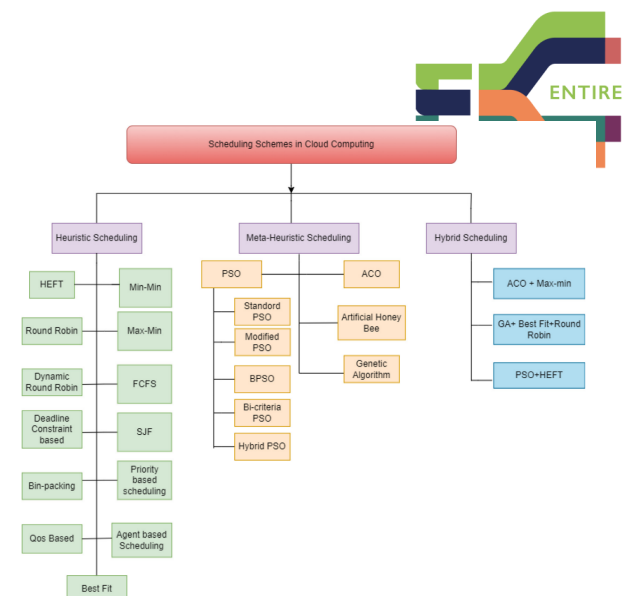
“Oh, bosh, as Mr. Ruskin says. Sculpture, per se, is the simplest thing in the world. All you have to do is to take a big chunk of marble and a hammer and chisel, make up your mind what you are about to create and chip off all the marble you don’t want.”-Paris Gaulois.

Source: <https://quoteinvestigator.com/2014/06/22/chip-away/>

- Heuristics
- Integer Programming
- Local search
- Deep neural networks

Heuristics

- Do not try to explore the search space
- Find a good enough solution by making greedy choices
- More general meta-heuristics schemes
- Very good heuristics exist for specific problem types
- Not compositional, added constraints may destroy existing approach
- Often not reusable code base

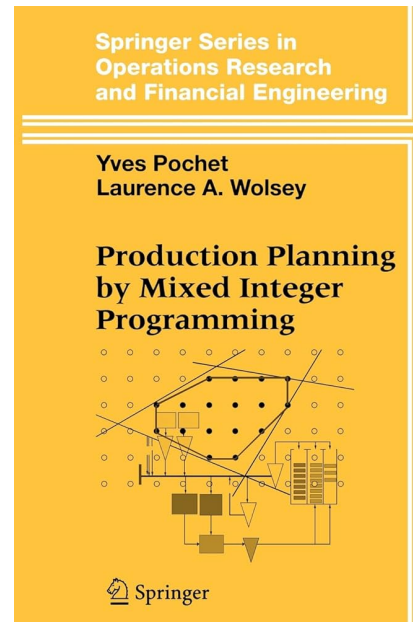


From: Singh, Kumar, and Singh: An empirical investigation of task scheduling and VM consolidation schemes in cloud environment, Computer Science review, 2023, <https://www.sciencedirect.com/science/article/pii/S1574013723000503>

Integer Programming



- Restrict yourself to linear constraints
- Powerful reasoning on the complete set of constraints
 - Linear Programming
 - Cut generation
- Expressing scheduling constraints can be difficult
- Scalability issues for detailed scheduling



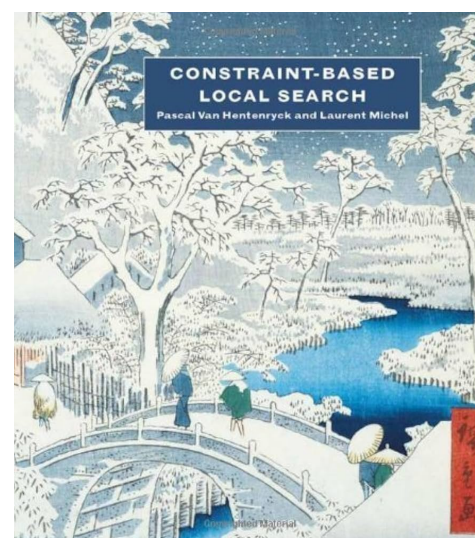
<https://link.springer.com/book/10.1007/>

0-387-33477-7

Local Search



- Start with an initial solution
- Try out changes that maintain feasibility
- Gradual improvement over time
- Not compositional
- No guarantee of solution quality
- Unifying approach:
Constraint-Based Local Search



<https://mitpress.mit.edu/9780262220774/>

constraint-based-local-search/

Course Structure (One Day Course)



Time	Topic
09:00-09:30	Registration & Welcome
09:30-10:30	Introduction & Motivation
10:30-11:00	Coffee
11:00-12:30	Scheduling Concepts
12:30-13:30	Lunch
13:30-15:00	Case Studies
15:00-15:30	Coffee
15:30-16:30	Hands-on Experience

What is not covered?



- How does it all work?
- How to integrate into an existing IT environment
- How to define and solve new constraints
- Interactive solving techniques

Are there LLM based scheduling approaches?



- Direct creation of solutions by LLM
 - Has been tried in multiple areas, does not seem to work well
 - Can you trust the solution (Hallucinations)?
 - Is it repeatable?
 - Can you understand what it is doing?
- Use LLM to create Constraint Program, then solve model
 - You can validate the generated model
 - Maintains performance gains by solver
 - Useful for prototypical data extraction from unstructured text
 - Does not replace proper workflow for production

How does it all work?



- You don't really need to know this to use Constraint Programming
- Advantage of declarative, compositional formulation
- I teach an introductory course on Constraint Programming for CRT-AI
 - Ask for details if interested
- Overview of courses, books and materials at <https://arxiv.org/abs/2403.12717>



- Why use Constraint Based Scheduling?
- Compared to other AI methods
- Compared to other solution approaches



Part II

Concepts



- We introduce the core concepts used in scheduling
- Different layers of description
 - What we are doing (jobs, tasks, resources)
 - Why we are scheduling (orders, products, processes)
- Temporal Relations
- Process description
- Problem classification
- Visualization

Most basic description of scheduling problem



- *Job*
 - Collection of activities required to manufacture one object/lot/order
 - Overall start/end determined by starts and ends of its tasks
- *Task*
 - Individual activities required for manufacture
 - Have defined start, end (typical: variables) and duration (sometimes fixed)
 - Often performed on one specific resource (more on that later)
- *Resources*
 - Resources are needed to perform the tasks
- Very compact representation of scheduling problem
- But, where does that information come from?



- An *order* specifies a need for a certain *product* at a given time in a specific quantity
- There may be multiple ways of making the *product* (multiple *processes*)
- We assume that the process to use is decided when placing the order
- Each order corresponds to a job, with its constituent tasks
- There may be limited visibility of future orders

Process Description



- Each *process* consists of one or more *process steps*
- A process step contains a duration formula to describe how long it lasts
- The order of *process steps* is defined by *process sequences*
- The resources needed are defined by *resource needs* (described later on)
- Tasks are created for each process step, their duration is based on the duration formula and order quantity

Where do the orders come from?



- Made to order
 - Each order is caused by a customer request
 - Defines due date, release date often implied
- Made to stock
 - Orders are satisfied from stock
 - Inventory control strategy decides when to make product
 - Often called stock orders
 - More complex variant integrates production planning and detailed scheduling
 - Example later in course

Temporal Relations



- Temporal constraints between tasks and/or jobs
- Defined by the manufacturing process
- In simple cases
 - A single sequence of process steps performed in that order
 - Each task must finish before the next one can start

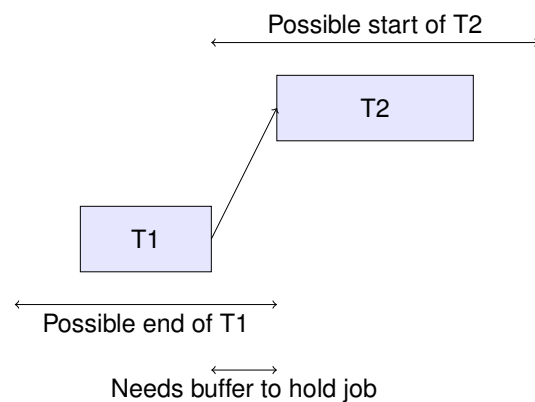


- ✓ Currently available in scheduling tool
- (✓) Will be available shortly
- ✗ Currently not available, may be added in future version

The Most Common Relation: EndBeforeStart ✓



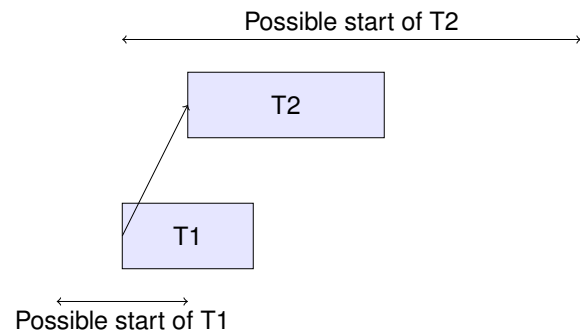
- States that one task (T1) must end before the next one (T2) can start
- Typical for manufacturing process based on the same item
- Addition: offset
 - Wait at least offset units between end and start
 - For example cooling, drying time outside a machine



Less Common: StartBeforeStart ✓



- States that one task (T2) can start any time after the start of another task (T1)
- Uncommon in manufacturing, occurs in project management
- Example later on on assembly line balancing

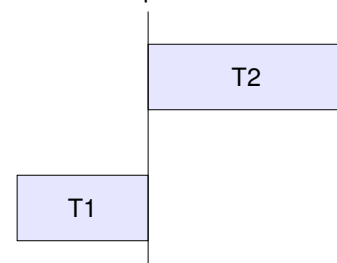


NoWait ✓



- Sometimes, two steps must follow each other immediately
- The item made would spoil
 - Product specific
- There is no space to hold item
 - Machine specific, buffers
- End of one task (T1) must be equal to start of next task (T2)
- May mean delay of start of task T1

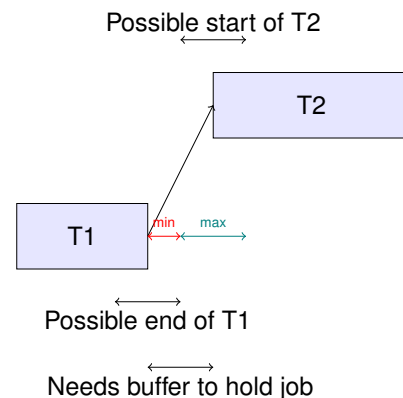
Start of T2 is equal to End of T1



MaxWait (✓)



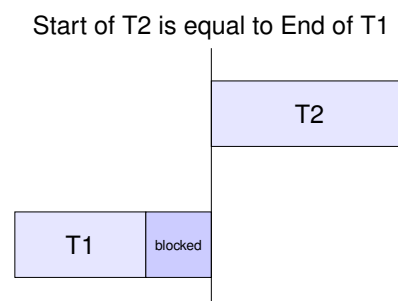
- Limit how long we can wait between tasks
 - Cooling enough, but not too much
 - Baking: rise time
- Impose both lower and upper waiting time limit
- Makes it more difficult to find solutions



Blocking ✓



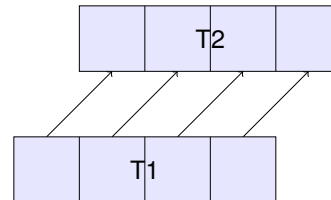
- Sometimes, two steps must follow each other immediately
- There is no space to store item between machines
- Keep item on previous machine until needed
- That machine is now *blocked*
- Duration of task T1 is extended until start of T2
- *Use with caution! Easy to deadlock*



Special Case: Pipelining X



- Sometimes, we can start on the next task while the first is still running
- Possible if one jobs produces multiple items (lots,...)
- As soon as the first item is finished, take it to the next machine to process it there
- Overlaps T1 and T2 as much as possible
- Details can get complex



More General: Relations between Intervals X



- First introduced by Allen (1983)
- 13 relations between intervals
- Allows composition of relations
- Constraint reasoning on sets of relations

Relation	Illustration	Interpretation
$X < Y$		X precedes Y
$Y > X$		Y is preceded by X
$X m Y$ $Y mi X$		X meets Y Y is met by X (<i>i</i> stands for <i>inverse</i>)
$X o Y$ $Y oi X$		X overlaps with Y Y is overlapped by X
$X s Y$ $Y si X$		X starts Y Y is started by X
$X d Y$ $Y di X$		X during Y Y contains X
$X f Y$ $Y fi X$		X finishes Y Y is finished by X
$X = Y$		X is equal to Y

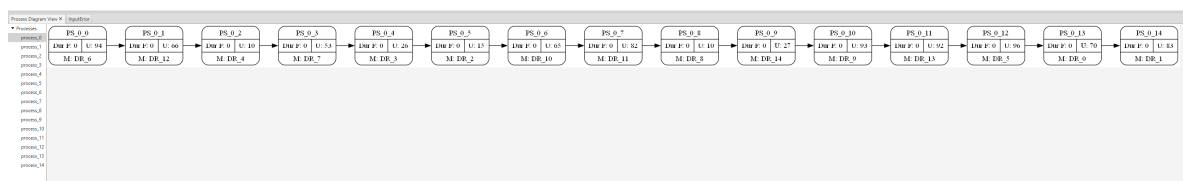
from Wikipedia: https://en.wikipedia.org/wiki/Allen%27s_interval_algebra

Start and End of Jobs ✓



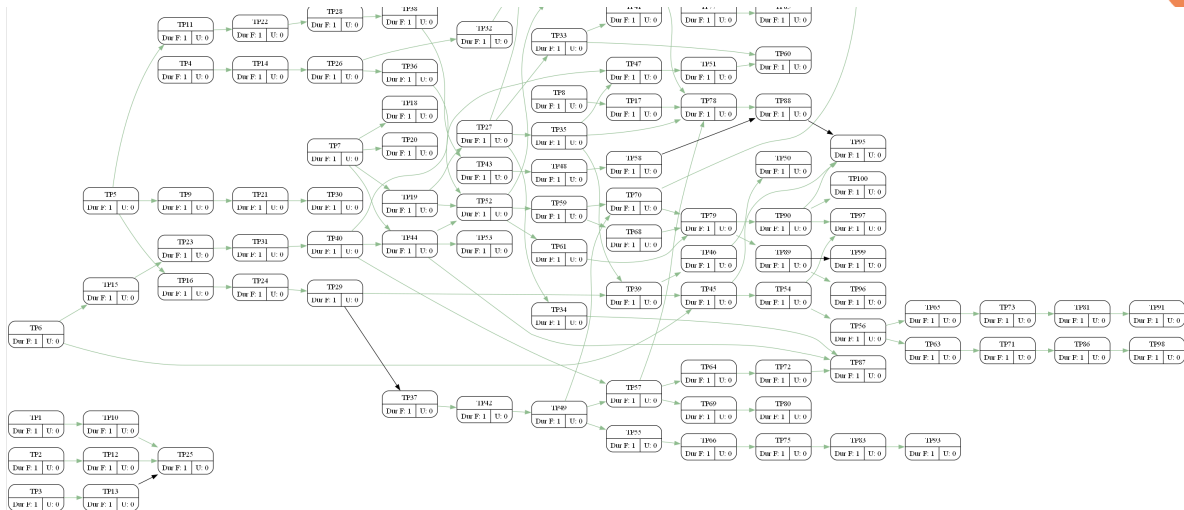
- The start of a job is equal to the start of the earliest task of the job
- The end of a job is equal to the latest end of any of its tasks
- Also called: the job *spans* its tasks
- Sometimes very simple
 - Start of job is start of first process step
 - End of job is end of last process step
 - But, do we know which steps will be first or last?

An Example of a Simple Process



- The steps form a precedence chain
- Easy to identify first and last step

An Example of a More Complex Process

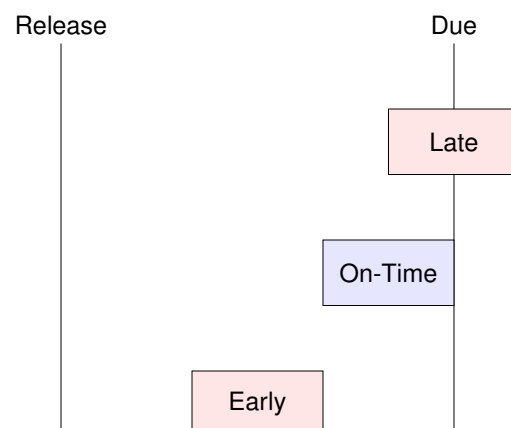


- There is no clear first or last process step

Jobs: Release and Due Dates ✓



- The execution of a job may be constrained in time
- *Release dates* states earliest time a job can start
- *Due dates* states latest time a job can end
- These may or may not be hard constraints!
- A job will be *late* if it ends after the due date
- A job will be *early* if it ends before the due date
- A job will be *on-time* if it ends at the due date



Relations between Jobs X



- There may be relations between jobs as well
- For example, jobs for the same product may be arranged by due date
- Do not allow to run job for a later due date before any job with an earlier due date
- Orders for the same customer, but different products, may be constrained
- Most common:
 - Jobs for intermediate products must finish in time for their use later on

More Complexity



- We have ignored a lot of potential complications
 - Alternative processes
 - Alternative process paths
 - Alternative resources
- Intermediate products
- Impact of raw material availability

Intermediate products X



- Some production operations are assembly steps
- Combine multiple intermediate products together
- These intermediate products need to be made as well
- There are processes for those products

Raw materials X



- Sometimes, a process step needs certain raw materials
- These are not made within the scheduled part of the plant
- They come from stock, inventory control problem
- Do we schedule production and then order raw materials?
- Do we schedule based on the available raw materials?

Bill of Materials (BoM), Bill of Processes X



- Enterprise systems will describe which items are needed to make a product
- Tree like structure, indicates the intermediate product/raw material needed and its quantity
- *BoM explosion* derive all required input materials for a given set of orders
- We may want to know at which step of process we need which materials (Bill of processes)
- This is where you use SAP, big database, trivial calculation
- Becomes hard if processes not fixed

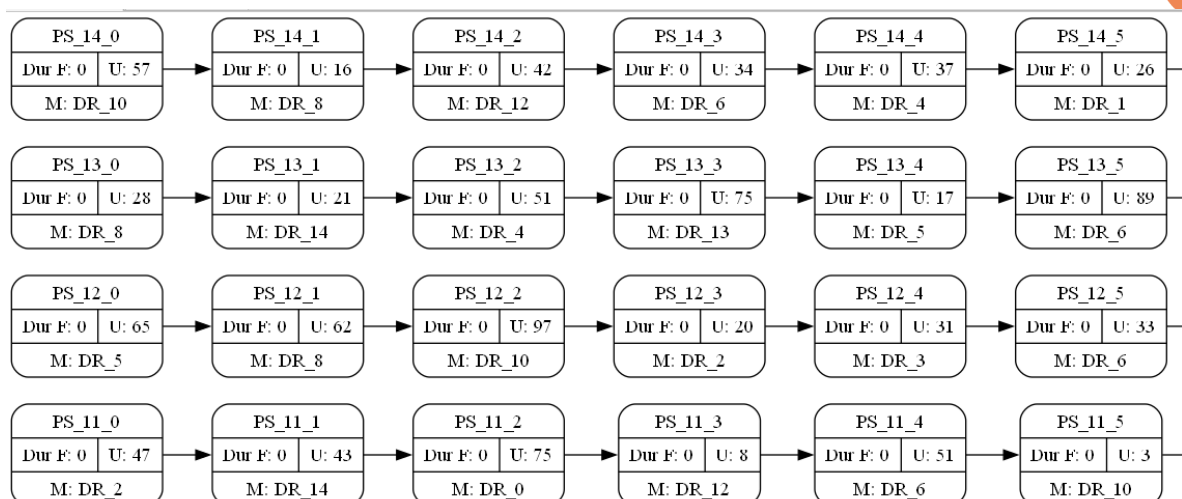
Problem Classification



- Most real-world problems are messy, with many special conditions and exceptions
- Academic research prefers well-structured problems
- Scheduling research often focuses on well-structured problem types
 - Easier to understand
 - Possible to exploit structure
 - Easier to compare results
- A small number of problem types are very common in research

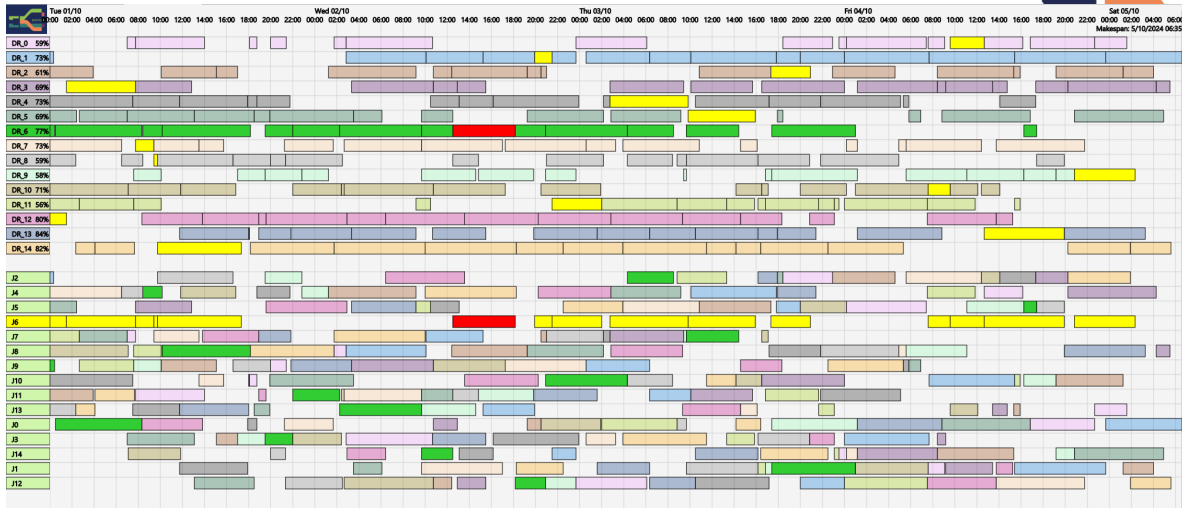
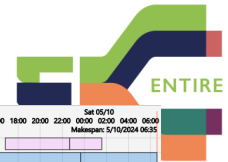
- Consists of a number of jobs and a number of machines
- Each job visits each machine, but possibly in a different order, depending on process
- Tasks of a job are linked as a precedence chain
- Objective is to minimize overall end, the *makespan*

Example Job-Shop Process



- Note that the order of machines visited is different for each process

Example Job-Shop Solution

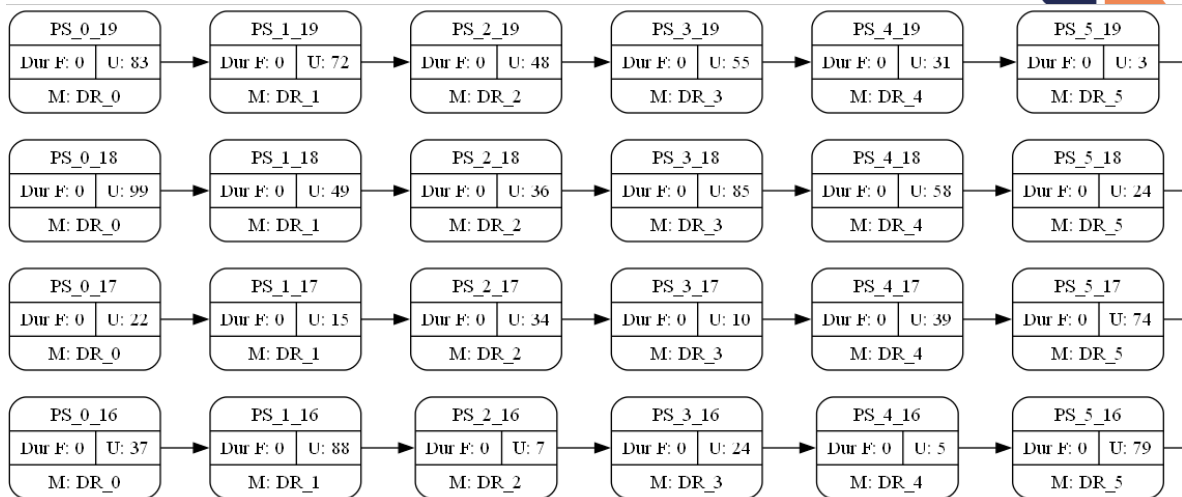


- One task is selected (in red), in both Machine and Job Gantt Chart
- Tasks are colored by machine, note coloring in jobs is different for each job

Flow-Shop ✓

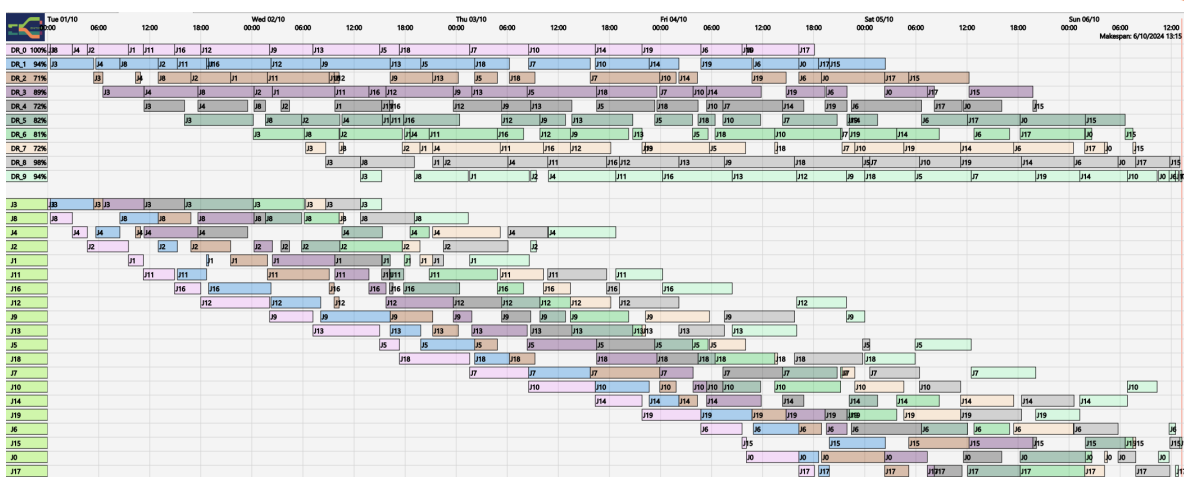


- Consists of a number of jobs and a number of machines
- Each jobs visits each machine, all jobs in the same order
- Tasks of a job a linked in a precedence chain
- Objective is to minimize overall end, the *makespan*



- Note that each process visits the machines in order DR_0, DR_1, ...

Example Flow-Shop Solution



- Tasks are colored by machine, note the regular pattern in the Job Gantt Chart

- Consists of a number of jobs and a number of machines
- Each jobs visits each machine, we have to choose the sequence individually for each order
- There are no temporal constraints between tasks, but tasks of the same job cannot overlap
- Objective is to minimize overall end, the *makespan*

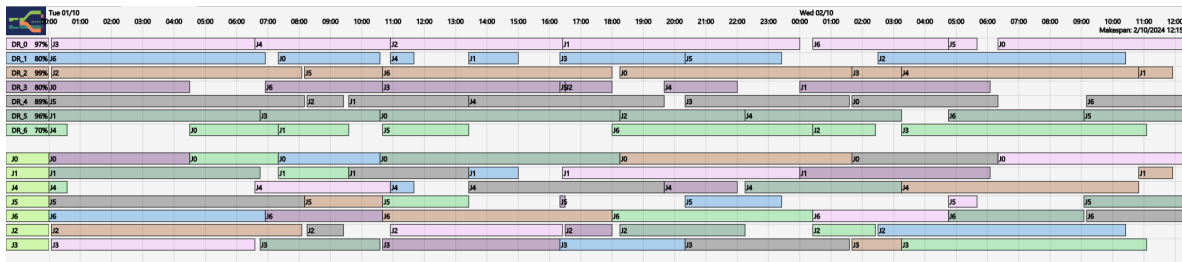
Open Shop Example Process



- Only showing details of one process
- No prescribed sequence between process steps
- Easier to find a task to run next
- Much larger search space

▼ Processes	
process_0	PS_0_6
process_1	Dur F: 0 U: 56
process_2	M: DR_4
process_3	
process_4	PS_0_5
process_5	Dur F: 0 U: 92
process_6	M: DR_5
	PS_0_4
	Dur F: 0 U: 71
	M: DR_0
	PS_0_3
	Dur F: 0 U: 34
	M: DR_6
	PS_0_2
	Dur F: 0 U: 54
	M: DR_3
	PS_0_1
	Dur F: 0 U: 39
	M: DR_1
	PS_0_0
	Dur F: 0 U: 89
	M: DR_2

Open-Shop Example Solution



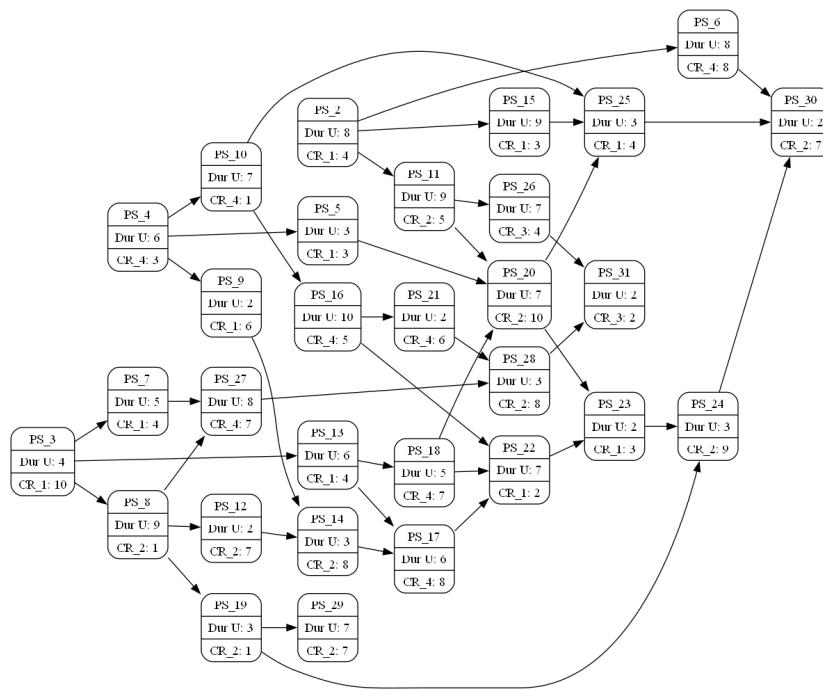
- Example solution for 7x7 open shop example
- Order of tasks within jobs not constrained
- Note that machines are still idle in optimal solution

Resource Constrained Project Scheduling Problem (RCPSP) ✓



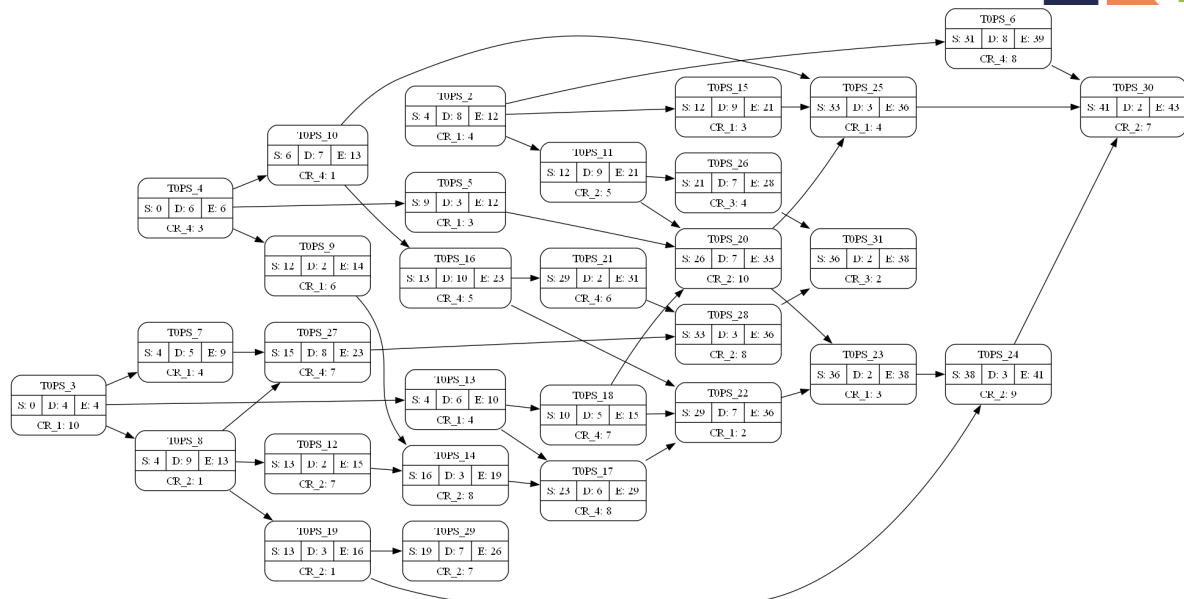
- Problem class from project management
- One project (one job), many tasks
- Precedence graph is arbitrary DAG
- Cumulative resources
- Variants with alternative modes, process alternatives ✗

RCPSP Process Diagram



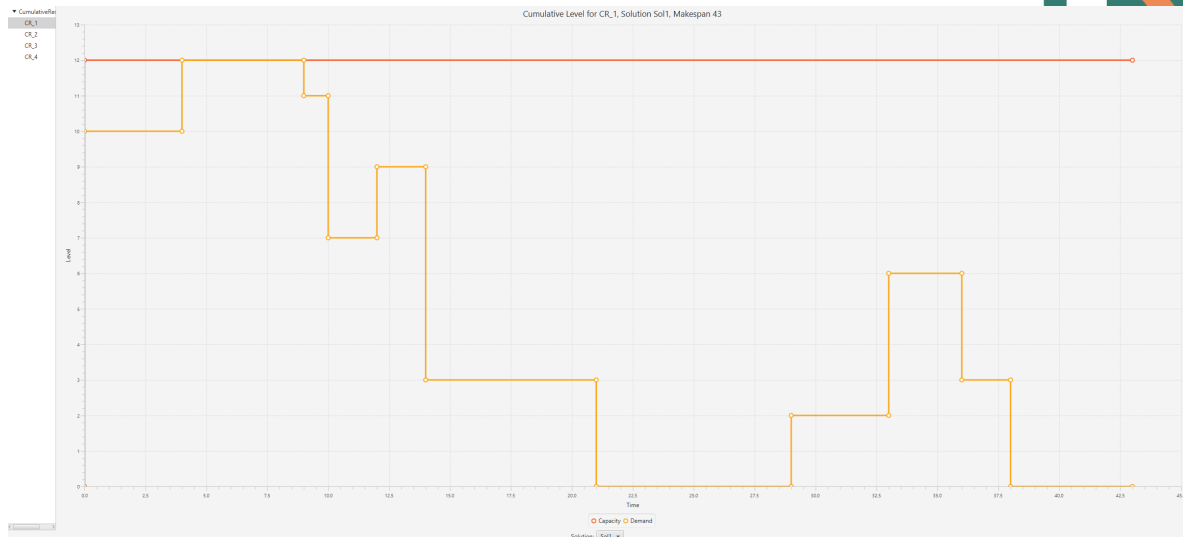
- Multiple sources, sinks
- Multiple cumulative resources

RCPSP PERT Diagram



- Resource constraints influence schedule, no single critical path

RCPSP Cumulative Chart



- Multiple cumulative resource, not busy all the time
- Not all instances are very hard to solve

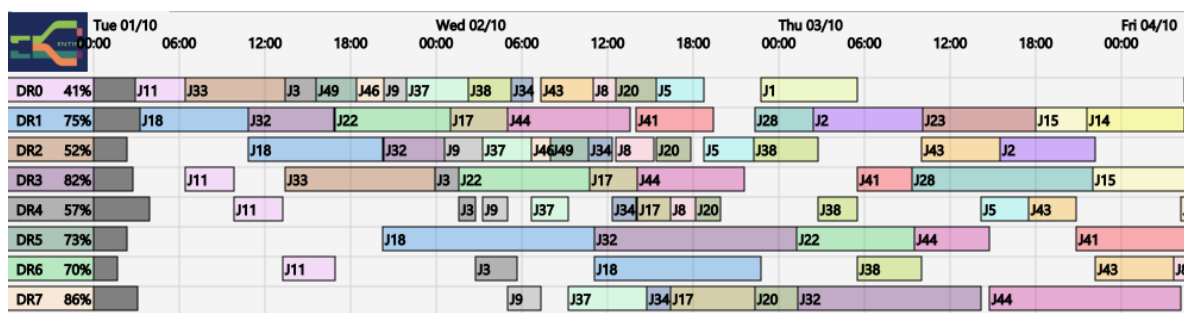
$\alpha/\beta/\gamma$ Notation



- The previous classes are good for algorithm research, but not very practical
- General scheme to describe problem type introduced in 1979
- Based on three parameters
 - α resource structure, stages
 - β temporal relations
 - γ objective
- $P2/r_j, \bar{d}_j/C_{\max}$: One stage, two identical parallel machines, hard release and due dates, objective makespan
- More detailed description at <https://encyclopedia.pub/entry/30497>

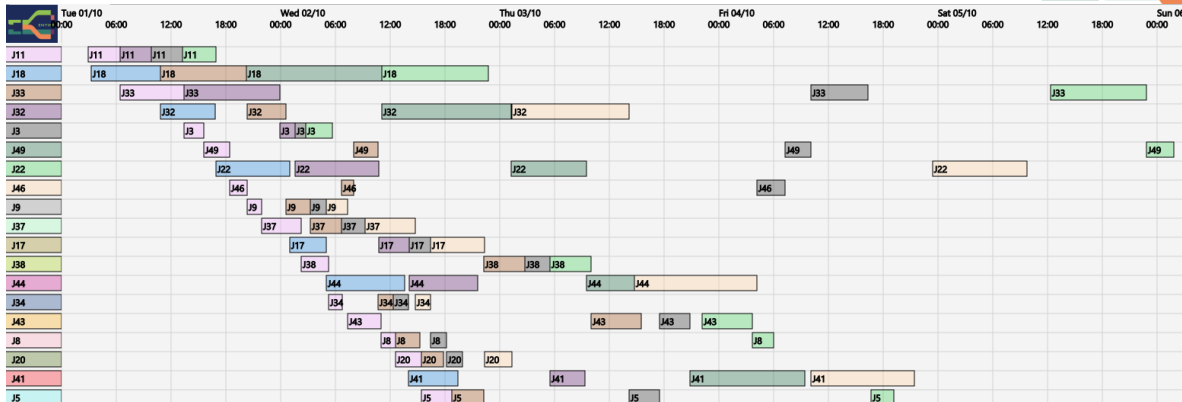
- Visualization is key to present and to understand results
- Many different ways to give an overview of schedule, and highlight problems
- Some diagrams types are used a lot, and are provided in our generic scheduling tool
- Customization is key

Machine Gantt Chart ✓



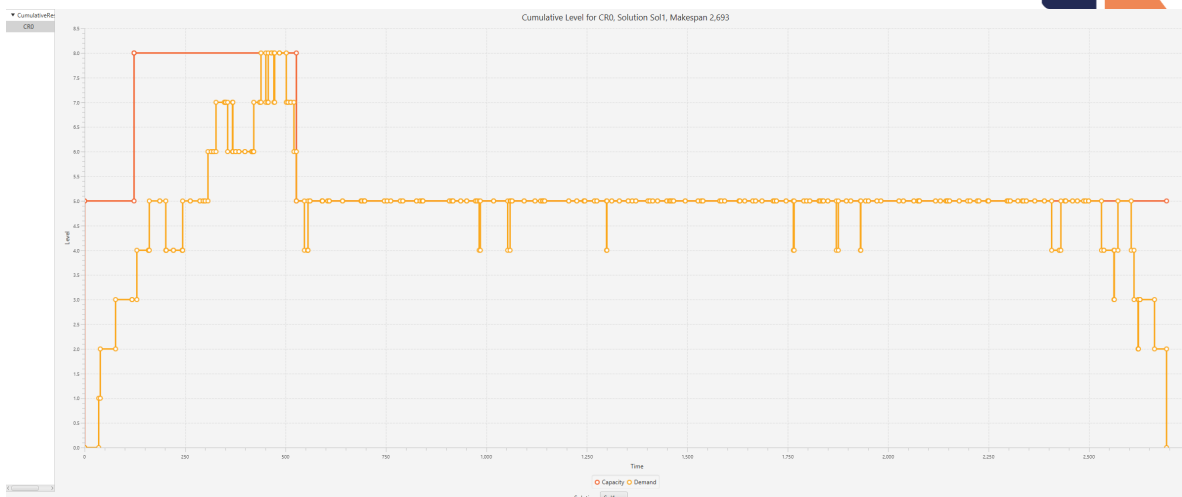
- Shows all tasks that are assigned to each machine
- Tasks should not overlap
- Also shows work in progress (WiP), down-times
- Optional display of setup and idle times

Job Gantt Chart ✓



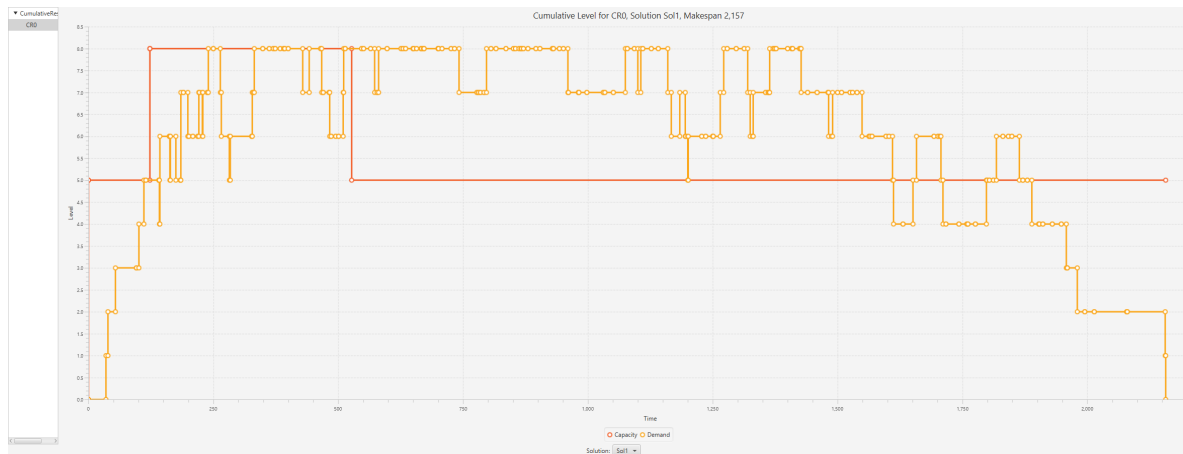
- Shows all tasks of a job in one line
- Only works for single chain of process steps
- Possible display of earliness, lateness
- Optional display of waiting and transport times

Cumulative Resource Chart ✓

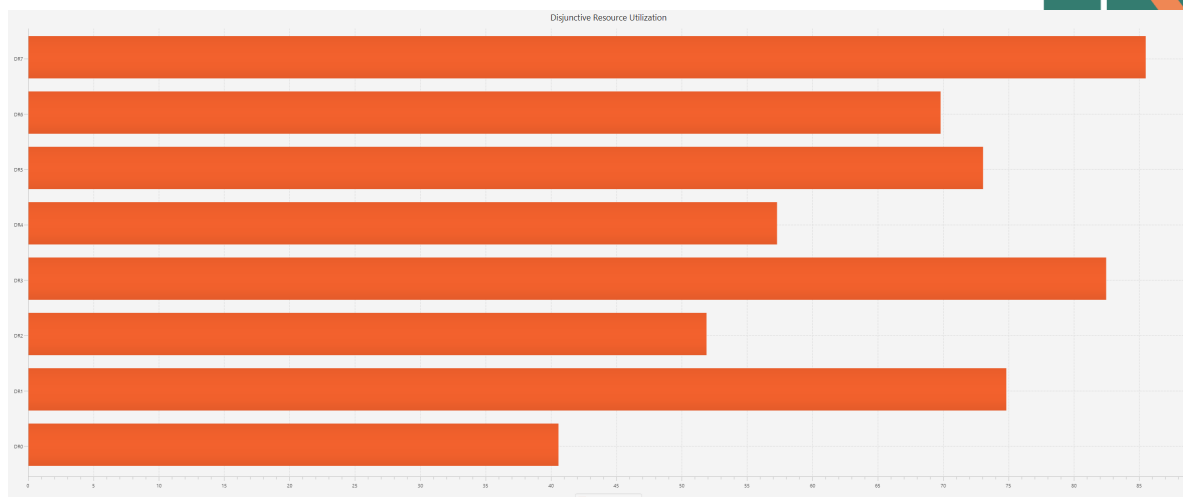


- Shows resource utilization of cumulative resource over time
- Utilization should be below capacity profile
- Unless we relax the cumulative resource constraint

Cumulative Resource Constraint Relaxed

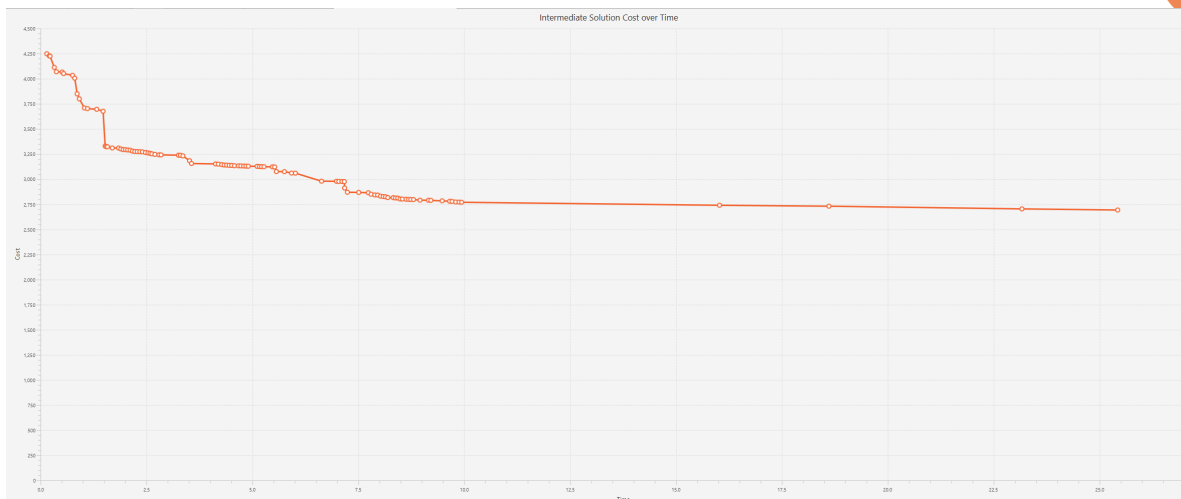


Resource Utilization ✓



- Shows utilization of machines as percentage of active time
- Helpful to identify bottleneck machines
- Information also shown in Machine Gantt

Intermediate Solutions ✓

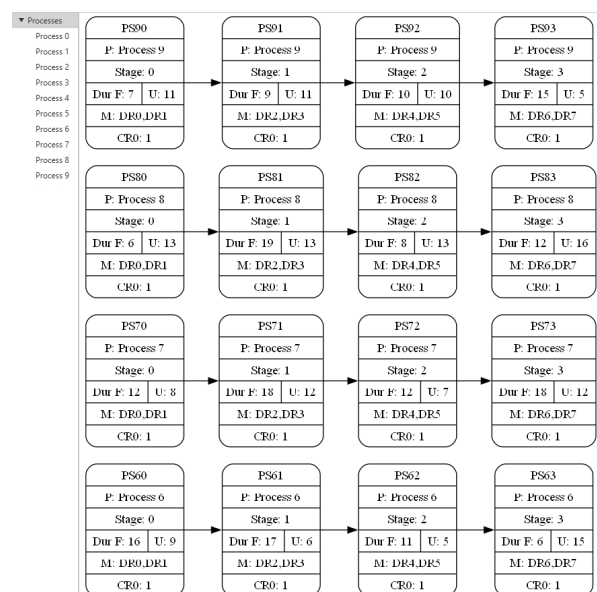


- Shows intermediate solutions found over time
- Useful to see if enough/too much time is allocated

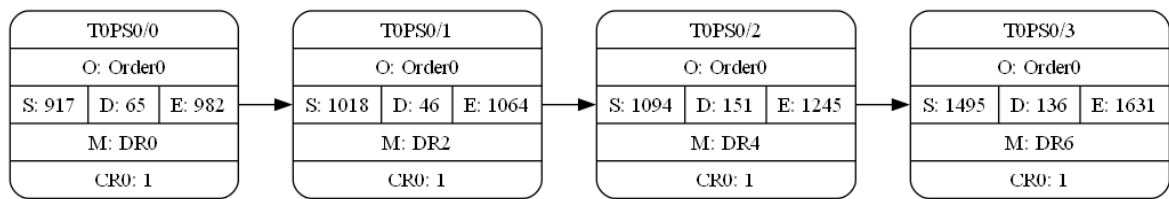
Process Diagram ✓



- See all details of one process in one image
- Can also look at all processes in one diagram
- Options to show/hide different fields

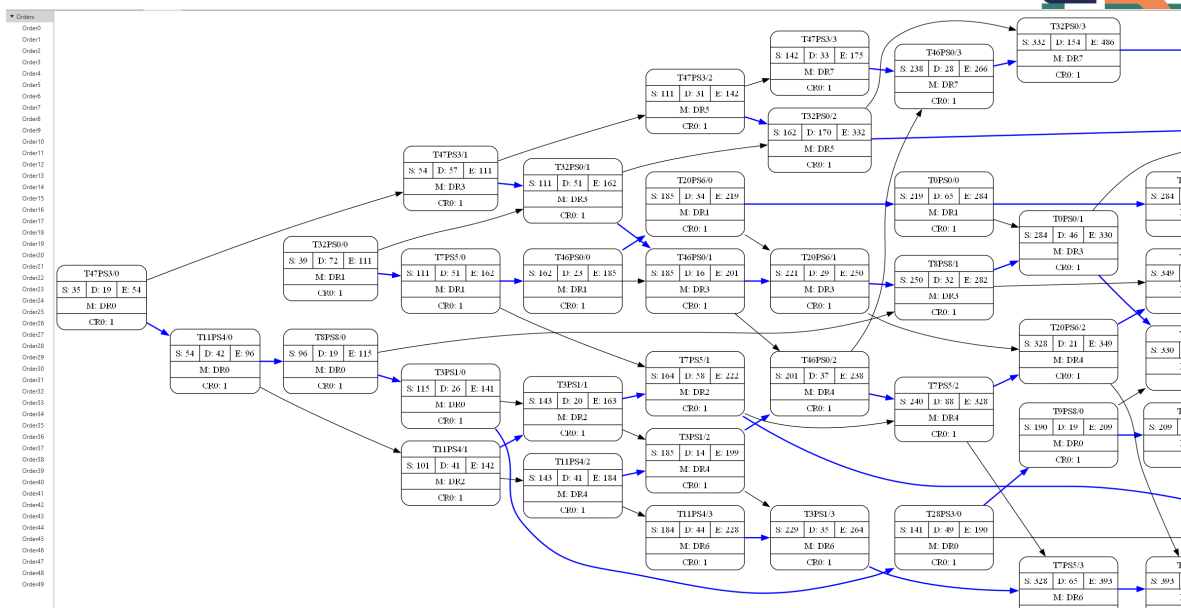


PERT Chart(Program Evaluation Review Technique) ✓



- Show details of job as a graph
- Useful if task graph is not a chain
- Often used in project management

PERT Charts become Confusing Quite Quickly



- Especially if all resource dependencies are included (in blue)

	Jan 22			Feb 22			Mar 22			Apr 22			May 22			Jun 22			Jul 22			Aug 22			Sep 22			Oct 22			Nov 22			Dec 22		
Mon				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	0	0	0	3	3	3	3					
Tue				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	0	0	0	3	3	3	3					
Wed				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	0	0	0	3	3	3	3					
Thu				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	0	0	0	3	3	3	3					
Fri				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	0	0	0	3	3	3	3					
Sat																																				
Sun																																				

- Shows weekly structure for one or more years
- Indicates public holidays, shut-downs, etc
- Indicating working days, KPI for each day

Summary

- We introduced the key concepts for scheduling problems
- Orders, products, processes
- Jobs and tasks
- Existing problem classifications
 - Academic
 - Limited practical usefulness
 - Used for benchmarking
- Key visualization ideas

Part III

Machines and Resources

ENTIRE EDIH

Scheduling (One Day Course)

Slide 102

Key Points



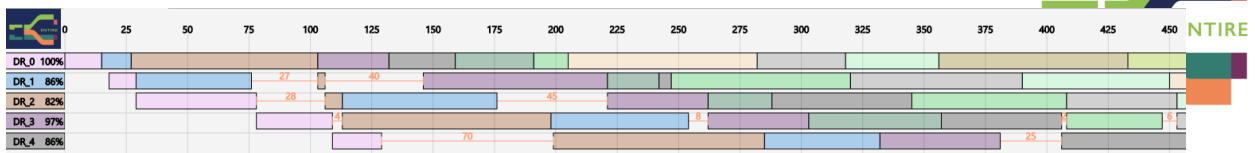
- Introduce different types of resources
- Disjunctive resources - one task at a time
- Cumulative resources - demands and capacity
- Machine choice - Use one of multiple machines
- Work in progress and planned downtimes
- Calendars - Not working all the time

Disjunctive Resource ✓

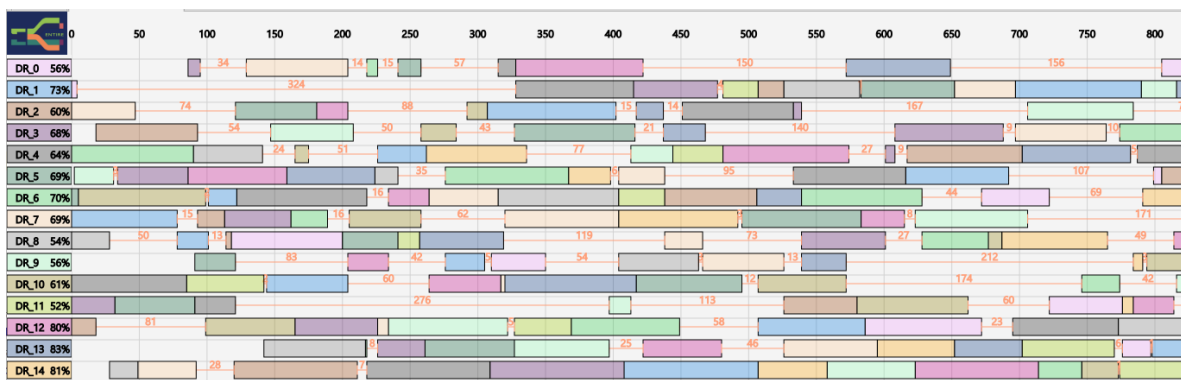


- A *disjunctive resource* works on one task at a time
- Each task runs uninterrupted from start to end
- The machine may be *idle* between tasks
- The machine may be unused at start and end of schedule
 - Some of this may be unreachable, there is no work that can be done in these periods
 - Problem of cold start, especially for flow-shop type problems
- *Active time* is time between first and last use
- Resource utilization compares productive time to active or available time

Disjunctive Machines Examples



- Flow-Shop example, some unreachable time on later resources in process, some idle time



- Job-Shop example, a lot of idle time

Preemption X



- Normal constraint for disjunctive constraints is one task at a time
- Once a task is started, it runs until it is finished
- *Preemption* allows to stop a task, run a different task, then resume the previous task to the end
- Example: This is how Operating Systems run tasks inside a computer
 - This works since cost of suspending a task is relatively low
 - Also needed as tasks continuously produce output which is expected
- In manufacturing, preemption often is an exception in an emergency
- Occurs a lot in project management, e.g. construction

How to Deal with Preemption in Scheduling



1. Handle this as manual intervention for critical situations
2. Dedicated preemptive scheduling constraints
3. Allow limited number of interruptions
 - Split each task into multiple pieces of unknown length
 - Normally, schedule all parts together for total duration
 - For preemption, schedule other task after first/second part
 - All parts of task must add up to total duration



- A cumulative resource provides capacity over time, the sum of the demands at each timepoint cannot exceed the available capacity at that time
- Resource demand by one task is considered constant from start to end
 - Need to break task into smaller segments to model time variable demand
- In itself a hard problem, so full propagation not possible
 - Active research area since 1993, when the constraint was introduced in CHIP

Specifying Cumulative Resources



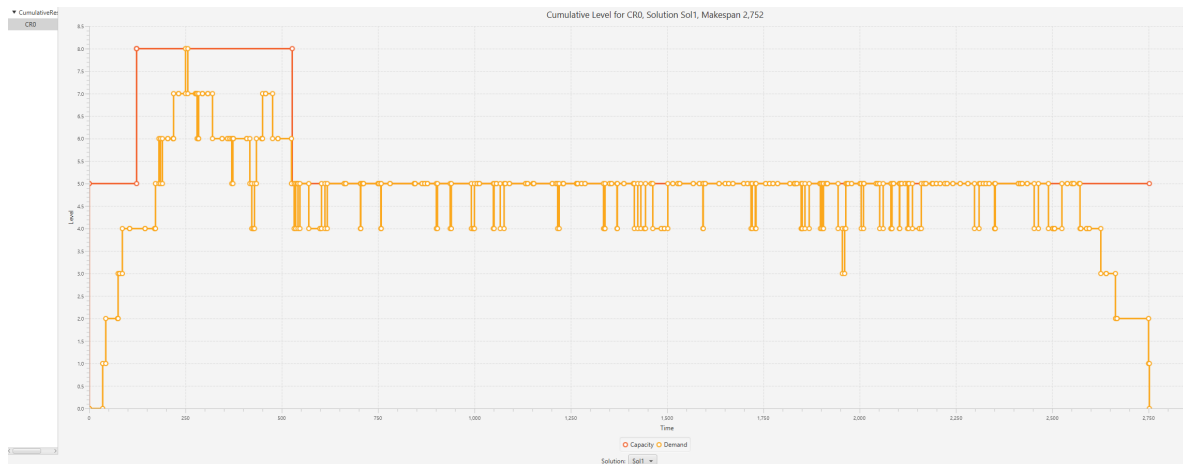
- Describing a cumulative resource
 - The resource itself
 - The capacity profile over time
 - The demands per processStep
- Each task may or may not need a specific cumulative resource
- The assumed total amount of work needed is constant
- We can calculate resource utilization by comparing demand to capacity

name	CR0
------	-----

fromDate	17/10/2024 07:16
cumulativeResource	CR0
name	CP00
from	0
capacity	5

cumulativeResource	CR0
name	CN0/0/CR0
processStep	PS0/0
demand	1

Cumulative Resource Profile



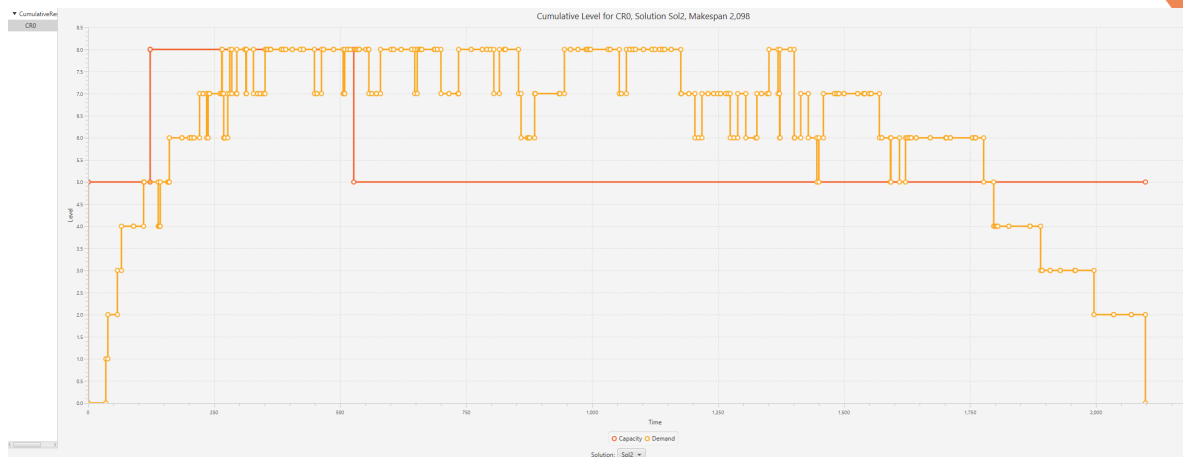
What is the Impact of the Cumulative?



- We want to understand what impact a cumulative resource has
- We can disable the constraint in the solver options
- Re-run the scheduler
- Observe the impact on the objective
- See where the capacity limit is not respected in new solution

Enforce Release Date:	☒
Enforce Due Date:	☐
Enforce Cumulative:	☐
Enforce WiP:	☒
Enforce Downtime:	☒
Enforce Setup:	☐
Enforce Transport Time:	☐
Relax Sequence:	☐
Add Same Order:	☐
Add NoWait:	☐
Add Blocking:	☐
Model Type:	CPSat
Solver Backend:	None
Objective Type:	Makespan

Cumulative Profile When Constraint is Disabled



- Objective reduced from 2,752 to 2,098
- Overall resource use now reaches 8 in period where capacity is limited to 5

Variant: Resource Limit as Objective X



- For some scheduling problems, the duration of the schedule is fixed
- The objective is: how many resources are needed to schedule all tasks within the available time?
- Capacity is a variable, part of objective function
- Example later on for assembly line balancing
 - Number of stations on line is fixed
 - Objective is to minimize *Takt*, the cycle time allocated for one step
- Consider solving this question with multiple scenarios, instead of different objective

Variant: Trading Time for Capacity X



- In some cases, the duration of a task depends on how many resources are available
- Total amount of work (energy) is constant, higher demand (power) means lower duration
- In easiest case, fixed demand levels are assumed
 - Resources are assigned to task throughout duration
 - Example: assigning software engineers to projects
 - Remember Brooks's law
 - *Adding manpower to a late software project makes it later.*
- Most general case, any profile is OK, as long total demand is covered
 - Cost of reassigning resource from one task to another is considered minimal

Variation: Time Variable Resource Cost X

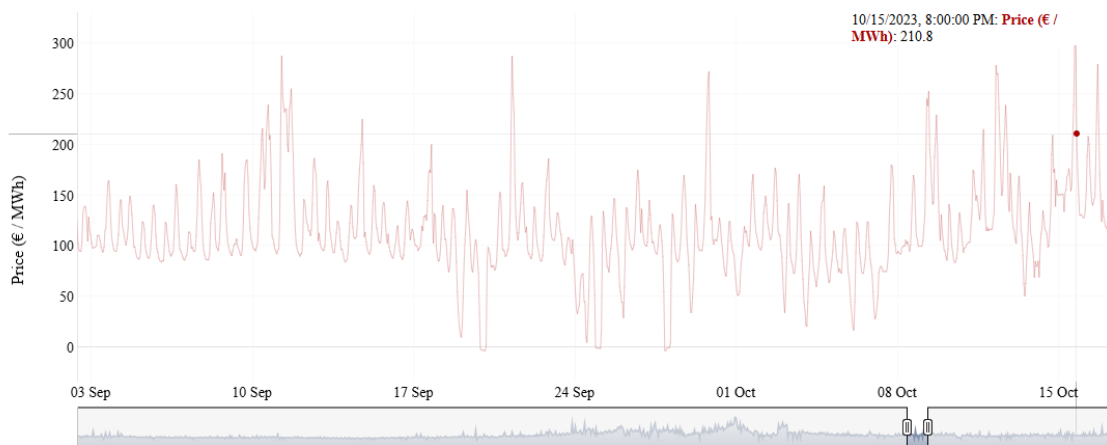


- Resource cost may vary over time
- Example: Overtime cost for working on weekend
- Example: Energy cost with time variable tariff
- Avoid periods of high cost, use areas of low cost

Example: Electricity Price in Ireland



Hourly Irish Wholesale Electricity Price



from: <https://kilowatt.ie/wholesale-electricity-prices-ireland/>

Variant: Soft/Hard Limits X



- Often, some capacity is available for "free", sunk costs
- Resource use above that limit costs extra
- Example: Full-time staff/contract workers
- Example: In-house capacity/rented computing capacity
- Multiple profiles, each with its own cost per unit

Variant: Lower Utilization Limit ✗



- Sometimes, we also want to enforce a lower limit of the resource use
- We want to avoid resources being idle
- Express a lower limit on the resource use
- Can be hard to satisfy for specific demand and capacity values

Manpower Constraints ✓



- Use cumulative constraints to express manpower limits
- Some tasks may need multiple workers
- Total capacity profile is number of workers available at each time
- Profile may change with shift-pattern (regular pattern)
- Holidays/sick-leave/training reduce available manpower at specific times
- Constraint does not assign workers, only checks that enough capacity is available



- Not all workers have the same qualifications
- Workers may need to be trained/certified to perform certain tasks
- Each task may require specific skill(s)
- Nested resource constraints to cover the needed skills
 1. One worker may have all required skills, only one worker is needed
 2. Multiple workers needed to cover all required skills, no worker has all skills
 3. More than one worker needed anyway, the group must cover required skills
- Training/certification program may create its own scheduling problem

Alternative: Assigned Operators X



- In special cases, it may be required to assign specific workers to tasks
- Each worker can work on one tasks at a time (disjunctive constraint)
- Multiple workers are qualified to perform certain jobs (machine choice, one worker is assigned)
- Multiple workers are qualified to perform certain jobs (multiple workers with that skill are needed)
- Named individual must be assigned for traceability
 1. Is there a hand-over from one shift to the next?
 2. Complete work must be performed within one shift



- Some tasks may not need a full-time operator
- Different scenarios
 - Operator only needed at start/end of task (setup, cleaning)
 - Operator is needed to load/unload items into machine
 - One operator can supervise three, but not four machines
- This gets too complex/too fragile very quickly

Choosing which machine to use



- Problem with Job-shop/Flow-shop: There is only one machine per processStep
 - What happens if any of those machines stops working?
 - Do we stop production completely?
- Most plants have multiple machine for the same task
- Three fundamental alternatives
 - Multiple, identical machines
 - Multiple machines with different speeds
 - Preferences for specific machines, but viable alternatives exist
- On the other hand, sometimes identical machines are treated as different
 - Dedicated lines for major products, avoiding setup/cleaning times

Identical Machines ✓



- Easiest case, several machines of same type
- You can choose any of the available machines
- Processing time is the same on all machines
- Product quality is identical
- Define which machines are available with ResourceNeed

Machine Dependent Speed (✓)



- Duration of the task depends on machine on which it is run
- Two common scenarios
 1. Some machines are faster than others (new generation)
 2. Different processes are faster/slower on some machines
- Express task duration as part of ResourceNeed
- Prefer faster machines, but balance machine use

Machine Preference (✓)

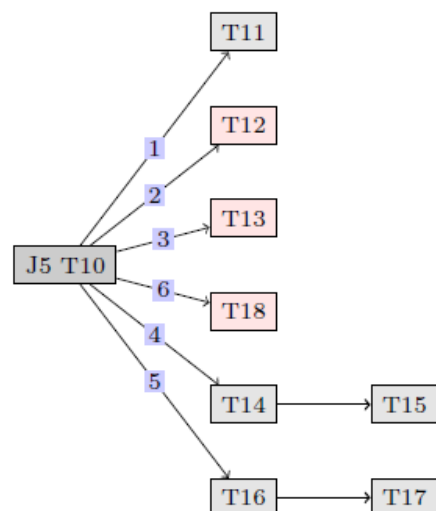


- Each process step has a preference ranking of machines, from best to worst
- Potential Causes
 - Product quality
 - Production speed
 - Production cost
 - Skill level required
 - Scrap rate
- Handle preferences as part of objective
- Enforce certain levels of preference to understand impact

Example from Siemens Energy Case Study



- Six alternatives for task T10
- Preference ranking from one (best) to six (worst)
- Some alternatives require additional tasks
- Tasks in red are outsourced



Work in Progress ✓



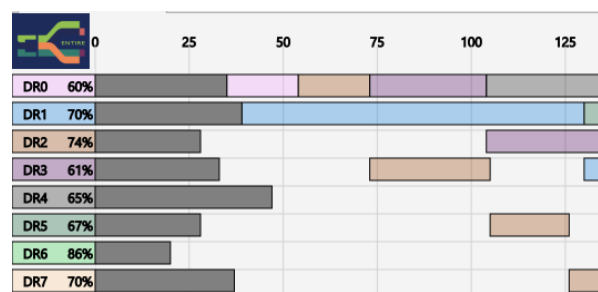
- Typical a plant does not start from scratch
- There is work currently running on machines
- This work must be finished before new work can be scheduled
- Called *Work in Progress (WiP)*
- Specified in input data

Name	DisjunctiveResource	Duration	Start	End	StartDate	EndDate
WDR0	DR0	35	0	35	1/10/2024 00:00	1/10/2024 02:55
WDR1	DR1	39	0	39	1/10/2024 00:00	1/10/2024 03:15
WDR2	DR2	28	0	28	1/10/2024 00:00	1/10/2024 02:20
WDR3	DR3	33	0	33	1/10/2024 00:00	1/10/2024 02:45
WDR4	DR4	47	0	47	1/10/2024 00:00	1/10/2024 03:55
WDR5	DR5	28	0	28	1/10/2024 00:00	1/10/2024 02:20
WDR6	DR6	20	0	20	1/10/2024 00:00	1/10/2024 01:40
WDR7	DR7	37	0	37	1/10/2024 00:00	1/10/2024 03:05

Work in Progress ✓



- Typical a plant does not start from scratch
- There is work currently running on machines
- This work must be finished before new work can be scheduled
- Called *Work in Progress (WiP)*
- Specified in input data, shown in gray
- Part of the disjunctive constraints



Planned Downtimes ✓



- Sometimes, a machine is unavailable for a period of time
- Maintenance, upgrade
- Planned activity with fixed start and end
- This should be considered in schedule
- Given as input data
- Part of the disjunctive constraints
- Gaps may lead to loss of productivity

Name	DisjunctiveResource	Duration	Start	End	StartDate	EndDate
DDR1	DR1	51	3,749	3,800	14/10/2024 00:25	14/10/2024 04:40
DDR2	DR2	66	5,137	5,203	18/10/2024 20:05	19/10/2024 01:35
DDR4	DR4	52	2,888	2,940	11/10/2024 00:40	11/10/2024 05:00
DDR6	DR6	57	4,412	4,469	16/10/2024 07:40	16/10/2024 12:25

Variant: Scheduled Downtime ✗



- Sometimes, we can decide when the downtime should occur (within reason)
- We can schedule it like any other task
- Avoid unproductive gaps in schedule
- More complex case for regular, scheduled downtimes
 - Maintain the correct time gap between maintenance checks
- How is in control in scheduling these events?

Unplanned Downtime X



- A machine breaks down unexpectedly
- This is not reflected in current schedule (unplanned)
- How to react?
 - Extend current task until finished (if task continues after breakdown)
 - Create new task to complete work later on (if task is partially finished)
 - Scrap task, reintroduce order in next schedule (if task is scrapped by breakdown)

Calendars



- A plant may not run 24/7, but shut down for regular/irregular periods
 - Overnight
 - Weekend
 - Public holidays/holidays/Christmas
- Some parts of plant may operate on different calendars
 - Office/lab may be working office hours only
- Considering multi-site problems, plants may be working in different time-zones
 - Common example: data centres around the world



- Which time points/time periods are expressed in working time, which in wall time?
- Examples
 - Release/due dates typically expressed in wall time
 - Task duration expressed in working time
 - Min/max waiting time expressed in wall time

Single, Factory-wide Calendar



- Three shift operation common
 - 06:00 - 14:00
 - 14:00 - 22:00
 - 22:00 - 06:00
- Start/end of weekend not obvious
- Handling of public holidays plant specific
- Lots of input data

Shift Pattern Definition X



Name	Shift Model	Percentage	Start Date	End Date	Start Time	End Time	Mon	Tue	Wed	Thu	Fri	Sat	Sun
AV12	Shift 15 1	0.80	01/02/2022	01/11/2022	06:00	14:00	x	x	x	x	x		
AV13	Shift 15 2	0.80	01/02/2022	01/11/2022	14:00	22:00	x	x	x	x	x		
AV14	Shift 15 3	0.80	01/02/2022	01/11/2022	22:00	06:00	x	x	x	x	x		
UV4	Shift 15 1	0.00	01/09/2022	30/09/2022	06:00	14:00	x	x	x	x	x		
UV5	Shift 15 2	0.00	01/09/2022	30/09/2022	14:00	22:00	x	x	x	x	x		
UV6	Shift 15 3	0.00	01/09/2022	30/09/2022	22:00	06:00	x	x	x	x	x		

- Definition of three shifts for Mon-Fri, shut-down in September
- Plant does not shut-down for Bank holidays (marked /)

	Jan 22	Feb 22	Mar 22	Apr 22	May 22	Jun 22	Jul 22	Aug 22	Sep 22	Oct 22	Nov 22	Dec 22
Mon												
Tue		3	3	3	3	3	3	3	3	3	3	3
Wed		3	3	3	3	3	3	3	3	3	3	3
Thu		3	3	3	3	3	3	3	3	3	3	3
Fri		3	3	3	3	3	3	3	3	3	3	3
Sat												
Sun												

- Resulting shift calendar

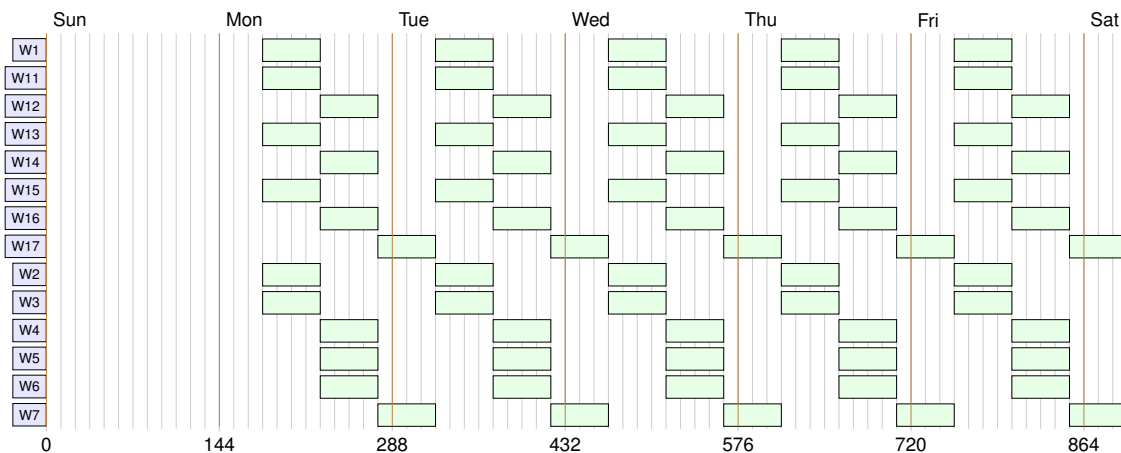
Weekly Machine Dependent Calendar



	Sun	Mon	Tue	Wed	Thu	Fri	Sat
A1							
A2							
BA							
BC							
BPCM1							
BPCM2							
E1							
E2							
F							
GR1							
GR2							
L1							
L2							
L3							
L4							
PM1							
PM2							
PW1							
PW2							
S1							
S2							
WC1							
WC2							
WC3							
WPCM1							
WPCM2							

- Note machines running one shift, two shifts, or three shifts

ShiftPattern for Workers



- Note different resource levels for morning, afternoon and night shift

Calendar Dependent Duration X

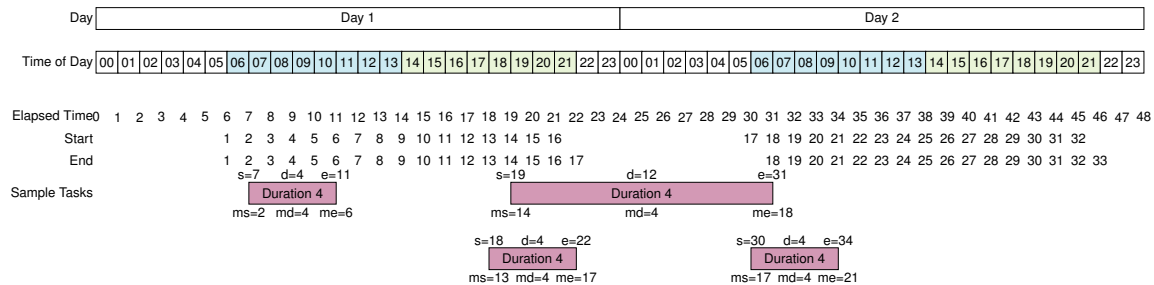


- In some factories, tasks have different duration depending on when they are run
- During the night-shift fewer workers are available, tasks like cleaning take longer
- During holidays, most expert operators are absent, tasks take longer due to less experienced operators
- For every working period, define a utilization factor to define nominal capacity (0-100%)
 - Tasks take longer if work capacity is lower
 - Only supported in few solvers (CPO)

Tasks Stretching over Shutdown X



- When a machine does not run continuously, the duration of tasks in wall time may change



- Task starting at 07:00 has duration of 4 hours
- Same task starting at 19:00 stretches over nightly shutdown (22:00-06:00), extending its duration to 16 hours

Summary



- Introduced different resource types
 - Disjunctive resources
 - Cumulative resources
 - Machine choice
- Identifying resources is a key element of defining scheduling problem
- Many problem specific variants exist, also impacting the constraint reasoning
- Keep as simple as possible - as complex as required
- Not all described variants already in our generic tool

Part IV

Objectives

ENTIRE EDIH

Scheduling (One Day Course)

Slide 149

Key Points



- Why we search for good, but not always optimal solutions
- The different objectives provided in scheduling tool
- More complex optimization schemes involving multiple objectives
- Other criteria that might guide which solution we prefer
- An interesting research direction

Why have an Objective?



- For most scheduling problem, we define some form of objective
- A mathematical formula that we evaluate on a schedule to compare it
- It is not always clear whether that formula represents some direct business benefit
- But, there are far more bad solutions than good solutions!
- The objective tells us if the solution is more "good" or "bad"
- Different stakeholders will have different views what makes a solution "good" or even "acceptable"

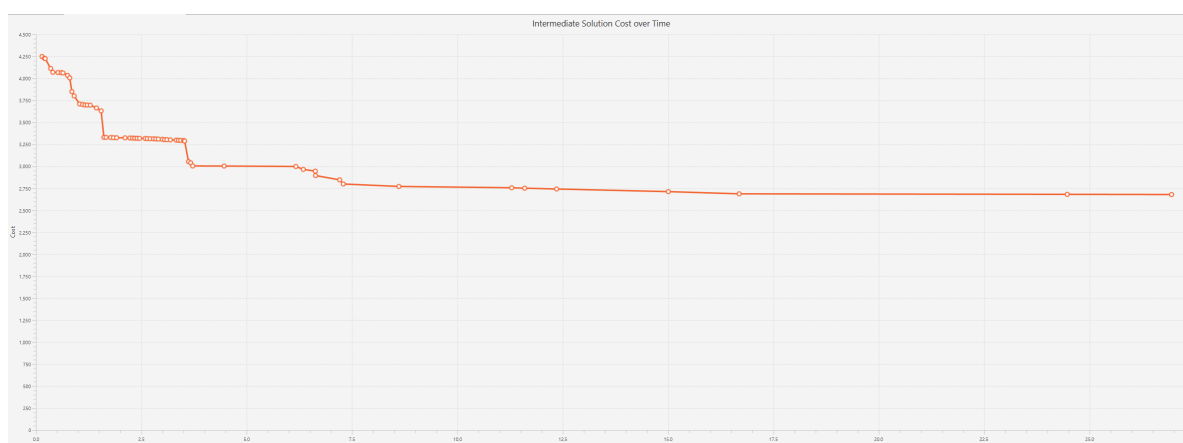
Minimizing Cost vs Maximizing Profit



- A lot of objectives aim to reduce cost of production
- This is not always a good thing
 - Doing nothing costs nothing
- But defining the profit obtained by a schedule is not easy
- Many intangible factors weigh in
 - Happiness of the customers (which customers are unhappy, does it matter?)
 - Happiness of personnel (finding and retaining skilled personnel is critical)
 - Happiness of stakeholders (sales, production, inventory, management)

- How quickly do we need a solution?
 - Sometimes we need a solution right now
 - We may also have time to wait a bit, or even more
 - Waiting five minutes, having a short break for a coffee, will often be acceptable
 - For some problems, running a scheduler overnight is possible
 - Do we need the ultimate in solution quality, or an acceptable solution right now?
- Benchmarks are often run with unlimited resources
 - "We used four years of computer time to solve these problems"

Diminishing Returns Running a Solver



- Which compromise between quality and speed are we looking for?

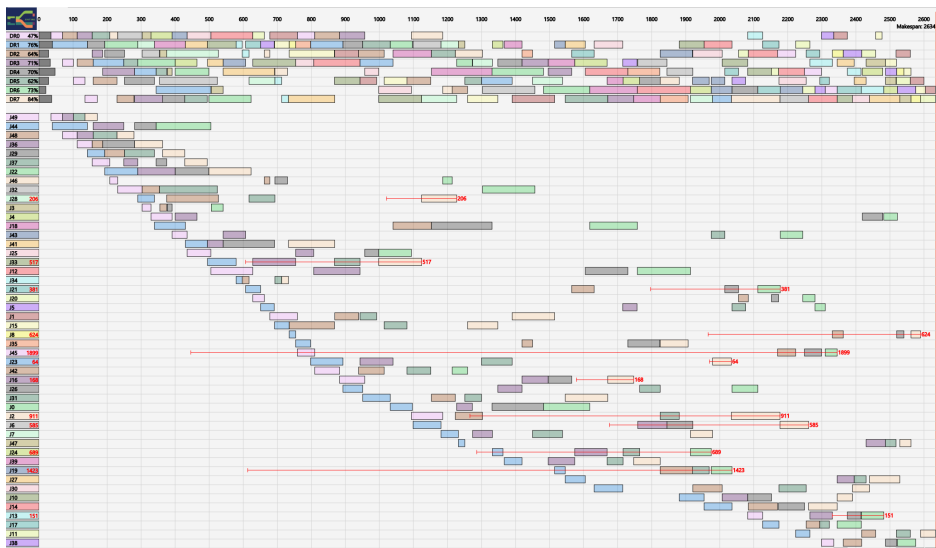
Setting the Objective



- We can select a predefined objective in solver dialog
- There are weight factors to give more impact to some cost terms in on-time and hybrid objectives

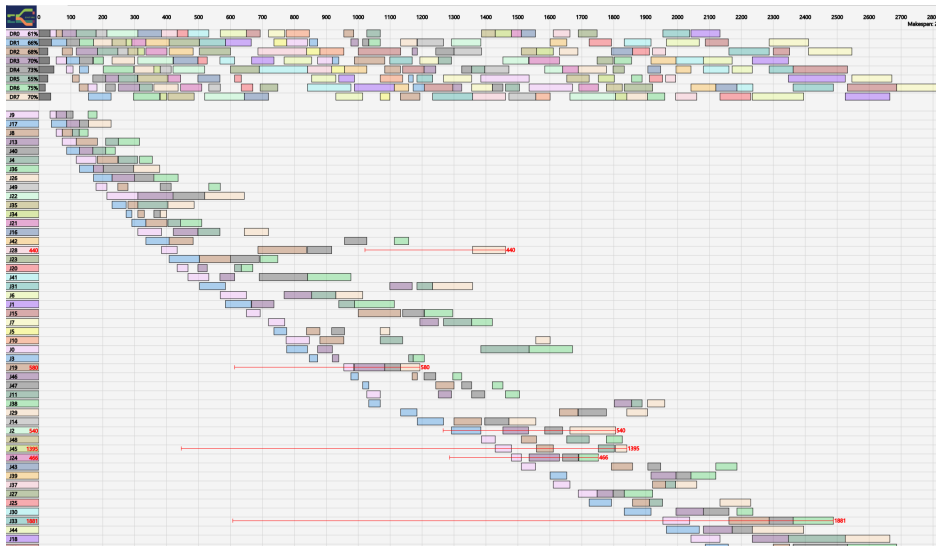
Objective Type:	Makespan
Weight Makespan:	1
Weight Flowtime:	1
Weight Lateness:	1
Weight Earliness:	1

Makespan ✓



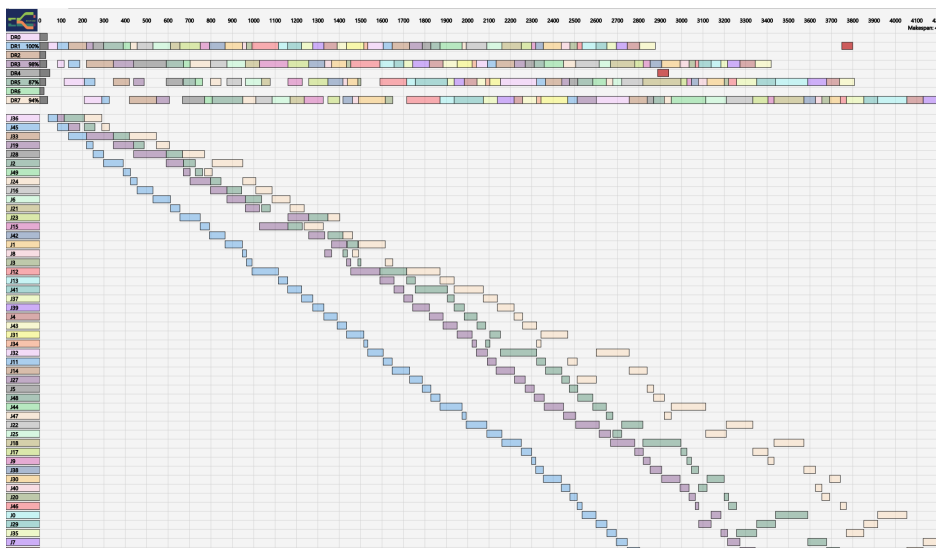
- Minimize the overall project end
- Very traditional objective in scheduling
 - Justified in project scheduling
 - Not so clearly justified in manufacturing
- A number of jobs are significantly late

Flowtime ✓



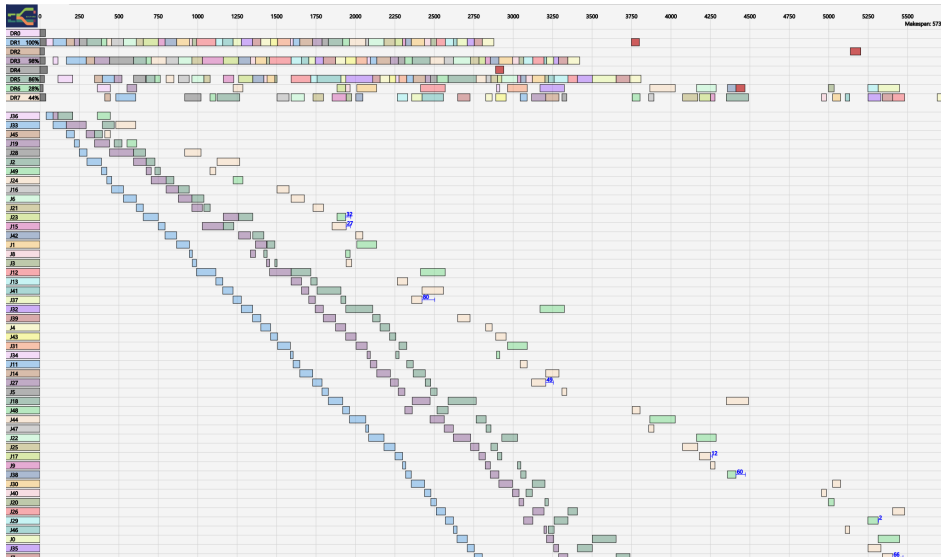
- Minimize the sum of job ends
- Prefer all machine to end early
- Not always easy to find good solutions

Total Lateness ✓



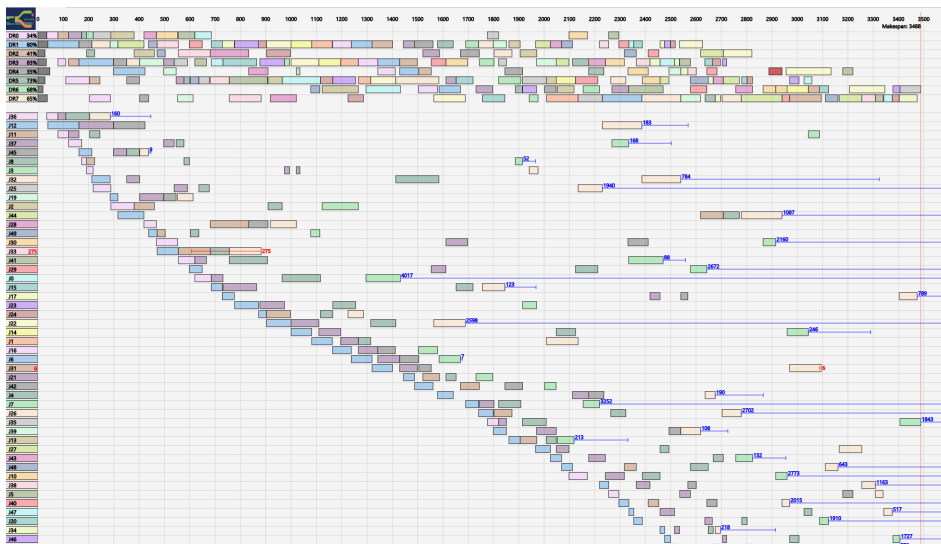
- Able to remove all delays on jobs
- Does not care about makespan or earliness
- In the example problem, many due dates far in the future

Maximizing On-Time Delivery ✓



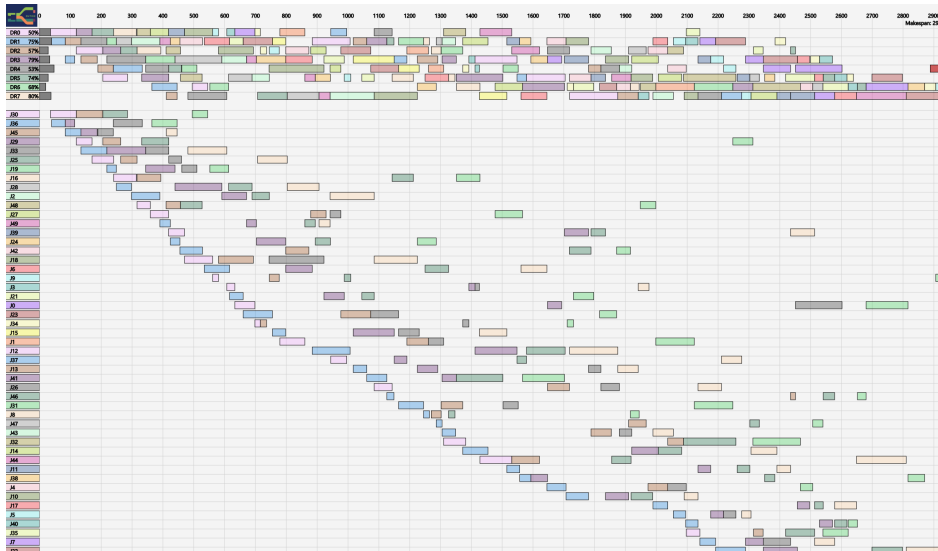
- Weight 100 for lateness, weight 1 for earliness
- Removes all delays, very little earliness
- Makespan increased dramatically

Hybrid Objective ✓



- Weights makespan:1000, flowtime:0, lateness:10, earliness:1
- Does not remove lateness completely
- Probably needs more time to improve

Hybrid Objective (Enforce Duedate)✓



- Sometimes enforcing a constraint is more powerful
- Here require that due dates are respected
- Leads to overall better solution

Comparing Solutions with Different Objectives



SolverRun	ObjectiveType	ObjectiveValue	SolverStatus	Bound	GapPercent	Makespan	Flowtime	TotalLateness	MaxLateness	NrLate	WeightedLateness	TotalEarliness	MaxEarliness	NrEarly	WeightedEarliness	PercentEarly	PercentLate
Run1	Makespan	2,634	Solution	1,050.00	60.14	2,634	86,339	7,618	1,899	12	7,618.00	76,688	4,887	38	76,688.00	76.00	24.00
Run2	Flowtime	66,356	Solution	39,248.00	40.85	2,842	66,356	5,575	1,881	7	5,575.00	94,628	5,045	43	94,628.00	86.00	14.00
Run3	TotalLateness	0	Optimal	0.00	NaN	4,239	119,745	0	0	0	0.00	35,664	1,494	50	35,664.00	100.00	0.00
Run4	OnTime	328	Optimal	328.00	0.00	5,733	155,081	0	0	0	0.00	328	80	8	328.00	16.00	0.00
Run5	Hybrid	3,554,610	Solution	1,150,697.00	67.63	3,488	117,180	281	275	2	281.00	38,510	4,017	34	38,510.00	68.00	4.00
Run6	Hybrid	2,992,627	Solution	1,155,981.00	61.37	2,934	96,782	0	0	0	0.00	58,627	4,530	43	58,627.00	86.00	0.00

- System tries to reduce the objective
- May mean other aspect of solution is poor
 - *Total Lateness* bad if just reducing *Makespan*
 - *Makespan* bad if just reducing *Total Lateness*
- Hybrid objectives can find better compromises
- Using constraints to restrict search can help
- Needs more work on lower bounds

Optimizing Resource Levels X



- We have already discussed this in the Resources section
- Sometimes we aim to optimize resource use, not time or delay
- Typical is minimizing
 - The number of disjunctive machines needed
 - A cumulative resource capacity
 - The manpower required to perform all tasks
- We may do this for understanding the problem
- The optimized schedules will be brittle
 - Any resource breakdown will cause an issue
 - Spare capacity is a good thing (if it is not too expensive)

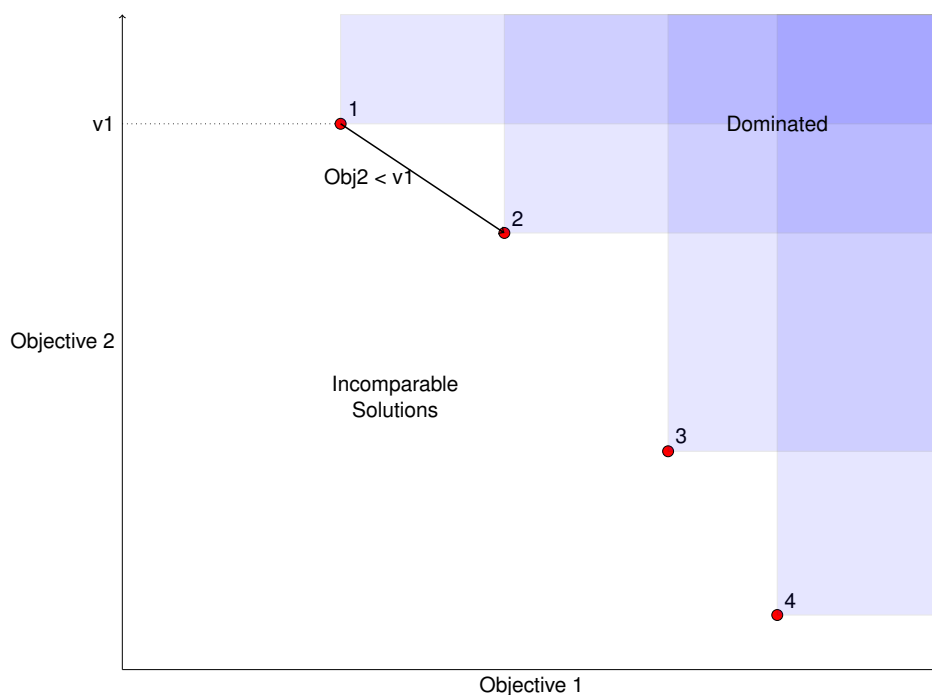
Including Optional Work X



- So far, the set of orders had to be fulfilled
- The resulting jobs needed to be included in schedule
- Sometimes, there are optional orders that we may or may not include in schedule
- The more work we do in this schedule, the better
 - Assumes a fixed horizon to limit the available space
- But we can reject some of the orders
- The objective is to maximize the value of the accepted orders
- Related to *Knapsack problem*, a well known combinatorial problem

- In some situations, a hybrid objective combining different aspects is not enough
- We need to find all best compromises between the different objective types
 - Without an a-priori weight to state which is more important
- A solution *dominates* another solution, if for all objective types, it is better than the other
- Two solutions are *incomparable* if for some objective type one solution is better, but for some other objective, the other solution is better
- *Pareto frontier*: Set of all non-dominated, incomparable solutions

Pareto Frontier for Two Incomparable Objectives



- Finding Pareto frontier by repeated optimization of objective 1



- There are other scales on which we may measure whether a solution is "good"
 - Fairness
 - Robustness
 - Product Quality
 - Customer Satisfaction
 - Diversity

Fairness



- Typically involves humans
- If we assign operators, do we
 - Treat all operators in a fair way?
 - Give effective workers always more work
 - Provide opportunities for training and skill development
 - De-risk dependency on key personnel
- Also, use multiple machines of same type consistently
 - Balanced
 - Not balanced



- By scheduling, we create a plan
- Often, reality does not follow the plan
 - Unforeseen events, machine breakdowns, sick-leave
 - Delays in raw material delivery, inventory problems
 - Rush orders
 - Small variations in plan execution
- Can we protect the plan against certain types of unplanned events?
- Is the plan still useful when things change?
- Or, can we update the plan quickly enough to adapt to changes

Product Quality



- The tighter the schedule, the more risk there is of cutting corners
- Example
 - If we minimize curing times to speed up production, quality may be affected
- The fastest machine is not always the best in terms of quality, cost



- Our objectives for minimizing lateness are lacking context
- Some customers are more important than others
- Some orders are more important to the customer than others
- A phone call by a human can capture more detail than an electronic order form
- We can adjust our schedule if we know what is important and what is not
 - But where do we get this information?
 - How do we avoid that a customer says "all my orders are critical"

Diversity



- Is it sometimes useful to present different solutions to a user to choose from
- These solutions should be substantially different to make choice meaningful
- Unfortunately, solvers often find very similar solutions
- Typically, there are far too many solution to enumerate them all
- We can add constraints to ask for the next solution to be quite different from the previous ones
- Needs good definition to define similarity
- Hamming distance on machine order a good starting point

Key Performance Indicators (KPI)



- Performance indicators can be computed from a given schedule, and allow to compare different schedules to each other
- Often, these are business oriented, not process driven
- There is a difference between an objective and a performance indicator
 - The objective drives the search for a solution
 - The KPI evaluates the quality of a solution, can be totally unrelated to objective
- Ideally, the KPI are expressed in such a way that solutions for different problems can be compared
 - Number of late orders, allows comparison of two solutions of the same problem
 - Percentage of late orders, allows comparison of two different schedules

KPIs for Sample Solutions



- Comparing different solutions of running example with enabling/disabling some constraints
- Compare *Makespan* to *On-time Delivery* objective
- There is no *Setup Time* constraint specified for this problem

Makespan	Flowtime	TotalLateness	MaxLateness	NrLate	WeightedLateness	TotalEarliness	MaxEarliness	NrEarly	WeightedEarliness	PercentEarly	PercentLate	Duration	Start	End		
2,688	83,425	10,083	1,959	11	10,083.00	82,067	4,938	39	82,067.00	78.00	22.00	2,653	35	2,688		
2,690	85,051	0	0	0	0.00	70,358	4,133	50	70,358.00	100.00	0.00	2,655	35	2,690		
2,136	58,403	0	0	0	0.00	97,006	4,956	50	97,006.00	100.00	0.00	2,101	35	2,136		
2,324	62,494	0	0	0	0.00	92,915	4,751	50	92,915.00	100.00	0.00	2,289	35	2,324		
5,733	154,918	0	0	0	0.00	491	122	10	491.00	20.00	0.00	5,538	195	5,733		
TotalWaitBefore	TotalWaitAfter	MaxWaitBefore	MaxWaitAfter	TotalIdleBefore	TotalIdleAfter	MaxIdleBefore	MaxIdleAfter	TotalSetupBefore	TotalSetupAfter	MaxSetupBefore	MaxSetupAfter	TotalActiveTime	TotalProductionTime	ActiveUtilization	SetupPercent	IdlePercent
23,297	23,297	1,943	1,943	6,823	6,823	435	435	0	0	0	0	19,917	13,094	65.74	0.00	34.26
24,903	24,903	1,611	1,611	5,901	5,901	342	342	0	0	0	0	18,995	13,094	68.93	0.00	31.07
12,081	12,081	449	449	785	785	80	80	0	0	0	0	13,879	13,094	94.34	0.00	5.66
0	0	0	0	4,211	4,211	111	111	0	0	0	0	17,305	13,094	75.67	0.00	24.33
0	0	0	0	28,641	28,641	773	773	0	0	0	0	41,735	13,094	31.37	0.00	68.63

KPIs Already Defined ✓



Makespan Max of job ends

Flowtime Sum of job ends

Total Lateness Sum of job lateness (tardiness)

Max Lateness Max of job lateness

NrLate Number of late jobs

WeightedLateness Weighted sum of job lateness

PercentLate percentage of late jobs

...Earliness same indicators, but for earliness

Duration Difference between overall start and overall end

Start start of earliest job

End end of last job

KPIs Already Defined (cont'd) ✓



TotalWait Sum of Wait time before/after a task of a job

MaxWait Max wait time before/after a task of a job

TotalIdle Sum of Idle times of disjunctive machines

MaxIdle Max Idle Time on a disjunctive machine

TotalSetup Total setup times

MaxSetup Max setup time

TotalActiveTime Total active time between first and last use of a machine

TotalProductionTime Sum of all task duration

ActiveUtilization Percentage of production time compared to active time

SetupPercent Percentage of setup time compared to active time

IdlePercent Percentage of idle time compared to active time



- If we have multiple solutions, we want to rank them based on a comparison of different KPIs
- Different stakeholders will rank different KPIs in very different way
- This seems to require some customization of the formulas used
- We can also try to infer a ranking method based on some comparison queries asked to users
 - Do you prefer this or that solution?
 - With enough answers, we can postulate a ranking method

Interactive Scheduling



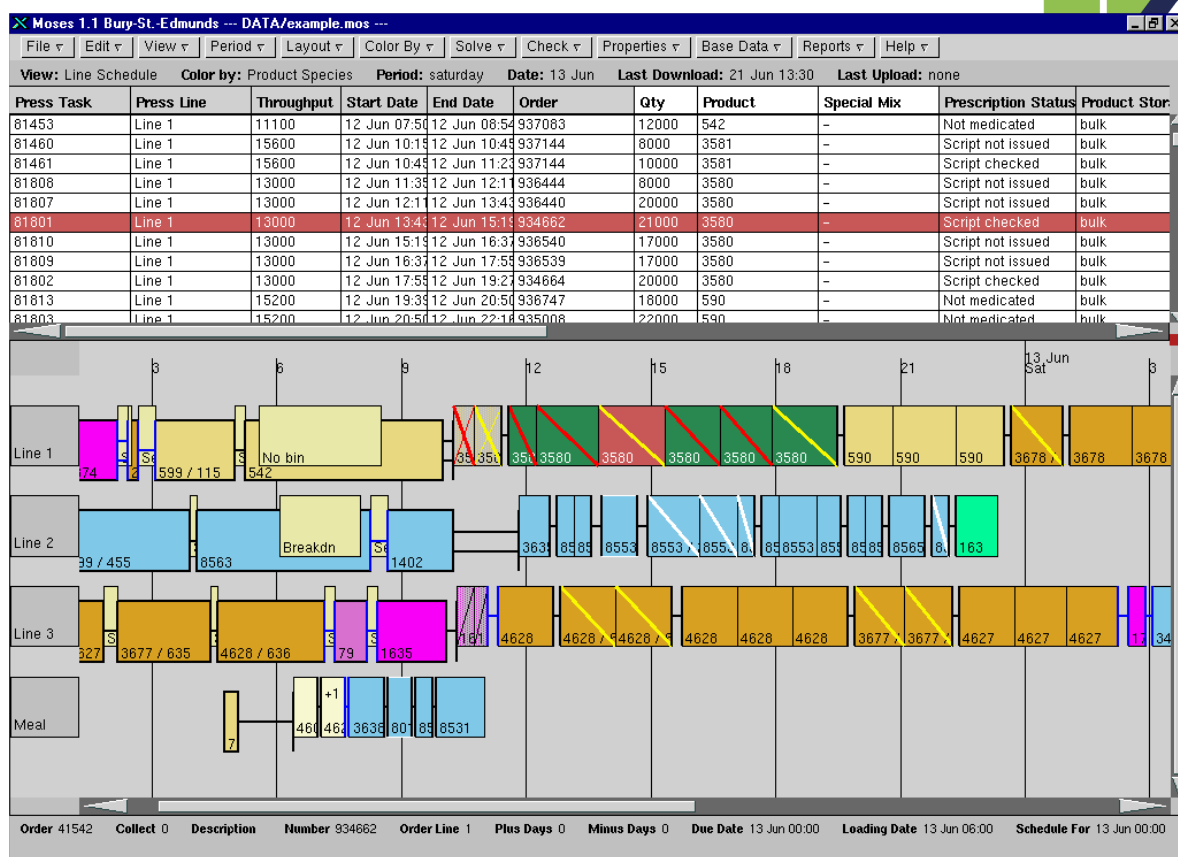
- Some human schedulers are happy to accept a produced plan
 - Perhaps change some constraints, or weights
- Other human schedulers want to modify the plan by hand
 - This is not always easy to do
 - How can a scheduling tool handle this?
 - How much control is given to the user, who checks the constraints?
 - Do we allow the user to create invalid schedules?

Example: Moses System



- Scheduling application for animal feed mills in the UK
- Produces overnight schedule for delivery on next day
- Operator updates the schedule whenever a task is finished
- Change duration of task if it is delayed
- Move tasks by hand, changing sequence of tasks to be performed
 - System updates constraints, and warns if constraint is violated
- User can protect part of schedule from modification by system
 - Freeze all tasks up to the selected task
 - Unfreeze the schedule after the selected task
- Related to explainability

Screenshot of Moses Application





- Describe the need and role of objectives
- Presented different objectives available in the scheduling tool
- Discussed some more advanced possibilities for handling objectives
- Important to keep user on control of system



Part V

Case Studies



- We provide a number of scheduling case studies
- Use the methodology developed to describe problems
- Use scheduling tool to provide solutions
- Generic tool provides good, but not always best solutions
- Two case studies are not handled by scheduling tool (yet)

Case Studies Overview



- ~~Production Planning and Detailed Scheduling~~
 - How to use detailed scheduling in a wider context
- ~~Assembly Line Balancing~~
 - Scheduling to plan design of an assembly line
- ~~Test Scheduling~~
 - Scheduling tests on resources
- ~~Factory Design~~
 - Location of resources affects scheduling outcome
- ~~Oven Scheduling~~
 - Solving one detailed scheduling problem is not enough
- ~~Blades and Vanes~~
 - Capacity and production planning over a multi-year period



- See how the methodology can be applied to solve real-world problems
- Generic tool provides immediate solution of good quality
- Visualization of results is also provided
- Tool is available on request



Part VI

Oven Scheduling Case Study



- Discusses two topics:
 - Solve a very specific industrial scheduling problem from the ASSISTANT EU project
 - Discuss the general issue of short-term scheduling vs. long-term objectives

Research Challenge



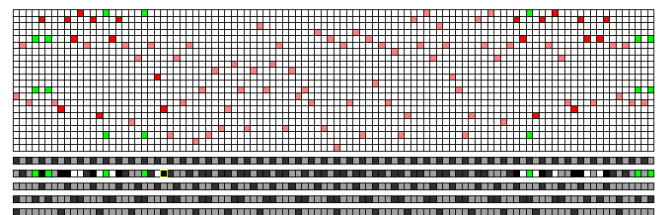
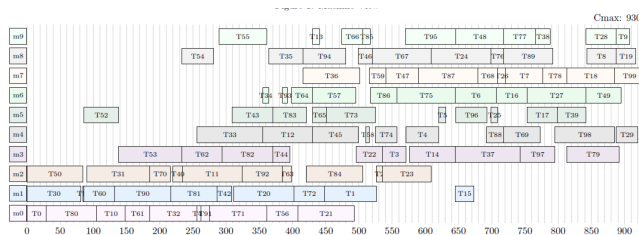
- Often the long-term business objectives are not visible in the operational decision problem
- We optimize a short-term objective without understanding the impact in the long term
- What choices should we make in short-term to improve overall result?
- Especially important when future data not yet visible
- Surprisingly, this problem is rarely discussed in literature

Examples



- Production Scheduling
- Nearly all scheduling benchmarks use C_{max} (makespan) as objective
- Why?
- Do we want to close factory as rapidly as possible?

- Car Sequencing
- The best heuristics push difficult cars to the edge of schedule
- Because they are easier to schedule this way
- But: It makes it hard to schedule next day



ENTIRE EDIH

Scheduling (One Day Course)

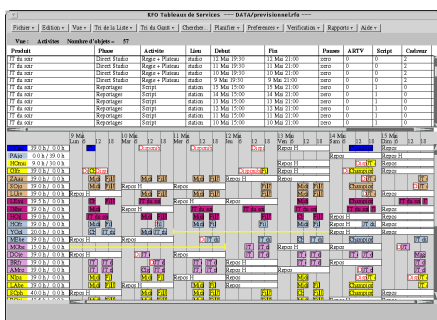
Slide 194

Examples



- Personnel Rostering
- Satisfy working rules and demands for period
- But: rules apply on a rolling horizon
- Easy to over-constrain problem for next period

- Transportation Planning
- Build daily delivery tours, optimizing cost
- Where are your trucks at 10PM?
- Also, avoid cherry-picking at start of week



ENTIRE EDIH

Scheduling (One Day Course)

Slide 195

Problem Studied Here

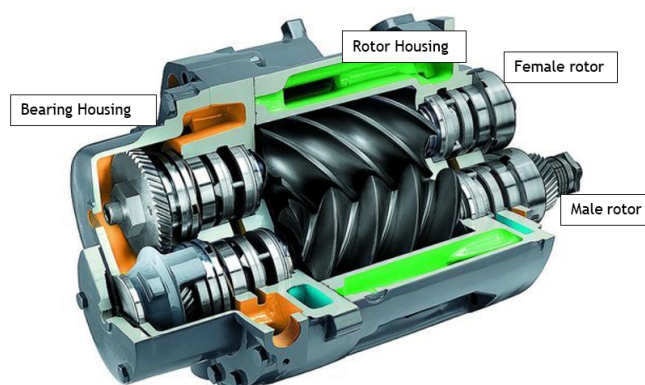


- Example from the ASSISTANT EU project (ended last year)
- Oven schedule for one of the industrial partners
- Schedule tasks on a set of ovens
- Tasks can share oven only if they are compatible
- Conflicting objectives
 - Energy use of ovens very significant, reduce when ovens are used
 - Waiting for an oven affects quality of product
- Jobs only visible when previous process step starts
- Currently scheduled by hand, industry partner expressed strong need for change

What does this look like in the real world?



Industrial Oven



Rotors in Compressor

- [illegible]

Slide 198

-
- The diagram shows a horizontal bar representing a decision horizon. The bar starts with a dark blue section on the left, labeled 'Commit', and transitions to a light blue section on the right, labeled 'Ignore'. The left end of the bar is labeled 'Certain' and the right end is labeled 'Uncertain'. A double-headed arrow below the bar indicates the 'Lookahead horizon'.

Slide 199



- Challenge: There is no global constraint to express the oven resource constraint
- We are not able to invest a lot of time/resources to develop such a constraint
- Two choices:
 - Two traditional models with variables linking them (Lackner et al, Constraints 2023)
 - Direct model expressing conditions as disjunctions of basic constraints

The Standard Pieces



- Jobs N consisting of multiple stages Q , tasks for each stage of each job, running on machines M
- Release dates r_i of jobs given by up-stream schedule
- WiP w_k on certain machines resulting from earlier schedule
- Machine m_{ij} and start variables s_{ij} for each task
- Precedence constraints between tasks of each jobs, with total waiting time c_i when waiting for resource
- Total number of ovens used in schedule $nrOvens$ by $nvalue$ constraint

$$nvalue(nrOvens, [m_{ij}|i \in N, j \in Q] ++ [k|k \in M \text{ s.t. } w_k > 0])$$

Resource Constraints



We start from the basic decomposition of the disjunctive machine choice constraint

$$\begin{aligned} \forall i_1, i_2 \in N \forall j_1, j_2 \in Q \text{ s.t. } \langle i_1, j_1 \rangle \neq \langle i_2, j_2 \rangle : \quad & m_{i_1 j_1} \neq m_{i_2 j_2} \vee \\ & s_{i_1 j_1} \geq s_{i_2 j_2} + d_{i_2 j_2} \vee \\ & s_{i_2 j_2} \geq s_{i_1 j_1} + d_{i_1 j_1} \end{aligned}$$

Express case where tasks share an oven (only when types and stages are the same)

$$\begin{aligned} \forall i_1, i_2 \in N \text{ s.t. } i_1 \neq i_2 \forall j \in Q : \quad & m_{i_1 j} \neq m_{i_2 j} \vee \\ & s_{i_1 j} \geq s_{i_2 j} + d_{i_2 j} \vee \\ & s_{i_2 j} \geq s_{i_1 j} + d_{i_1 j} \vee \\ & (t_{i_1 j_1} = t_{i_2 j_2} \wedge m_{i_1 j} = m_{i_2 j} \wedge s_{i_1 j} = s_{i_2 j}) \end{aligned}$$

Limit stacking

Need binary variables $b_{i_1 i_2 j}$ to state that two jobs i_1 and i_2 share oven in stage j



$$\begin{aligned} \forall i_1, i_2 \in N \text{ s.t. } i_1 < i_2 \forall j \in Q : \quad & (b_{i_1 i_2 j} = 0 \wedge (m_{i_1 j} \neq m_{i_2 j} \vee \\ & s_{i_1 j} \geq s_{i_2 j} + d_{i_2 j} \vee \\ & s_{i_2 j} \geq s_{i_1 j} + d_{i_1 j})) \vee \\ & (b_{i_1 i_2 j} = 1 \wedge t_{i_1 j_1} = t_{i_2 j_2} \wedge m_{i_1 j} = m_{i_2 j} \wedge s_{i_1 j} = s_{i_2 j}) \end{aligned}$$

Count how many jobs share stage j with job i

$$\forall i \in N \forall j \in Q : \quad z_{ij} = \sum_{i_1=1}^{i-1} b_{i_1 i j} + \sum_{i_2=i+1}^n b_{i i_2 j}$$

Limit how many tasks can be stacked together

$$\forall i \in N \forall j \in Q : \quad z_{ij} < \text{maxStacked}$$

This should not work!



- Weakness of basic decomposition model was the reason to develop the scheduling constraints in the first place
- Does not scale well to thousands of tasks
- But model is well suited to some solvers
 - SAT based solvers, Chuffed, CP-SAT (OR-Tools)
 - MIP solvers
- This works (only) as long as problem size stays manageable

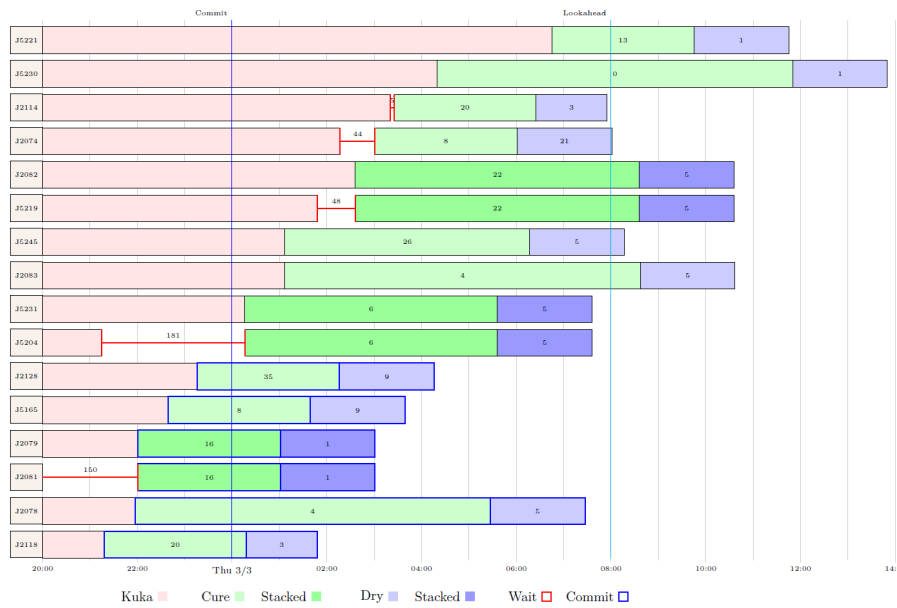
Compound Objective



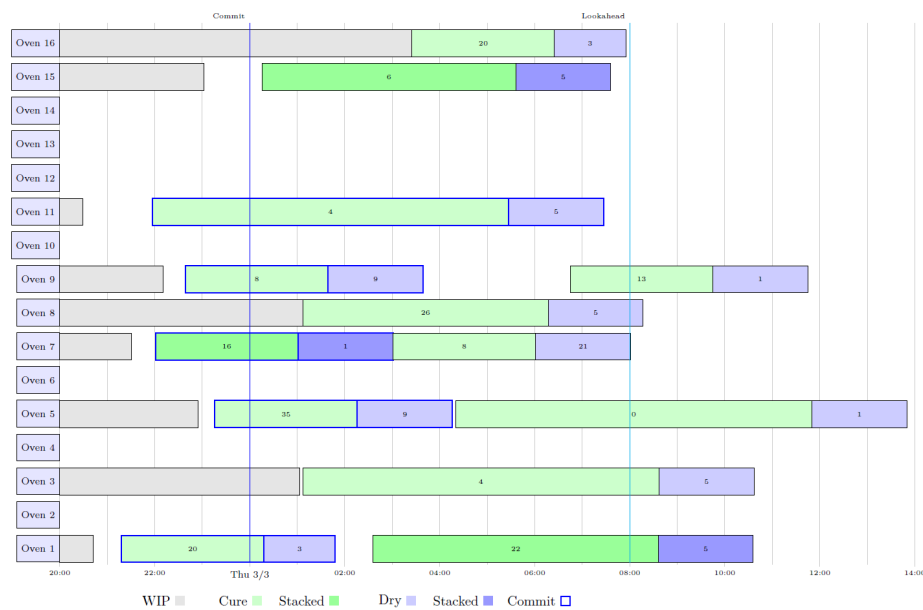
$$\min \alpha_1 \sum_{i \in N} c_i + \alpha_2 \text{nrOvens} + \alpha_3 \sum_{i \in N, j \in Q} z_{ij}$$

- Three conflicting elements
 - Total waiting time for jobs
 - Number of ovens used
 - Number of tasks stacked (negative coefficient)
- Reducing waiting time requires using more ovens
- Improved stacking will require for one job to wait until second is ready

Short-Term Schedule: Job View



Short Term Schedule: Resource View

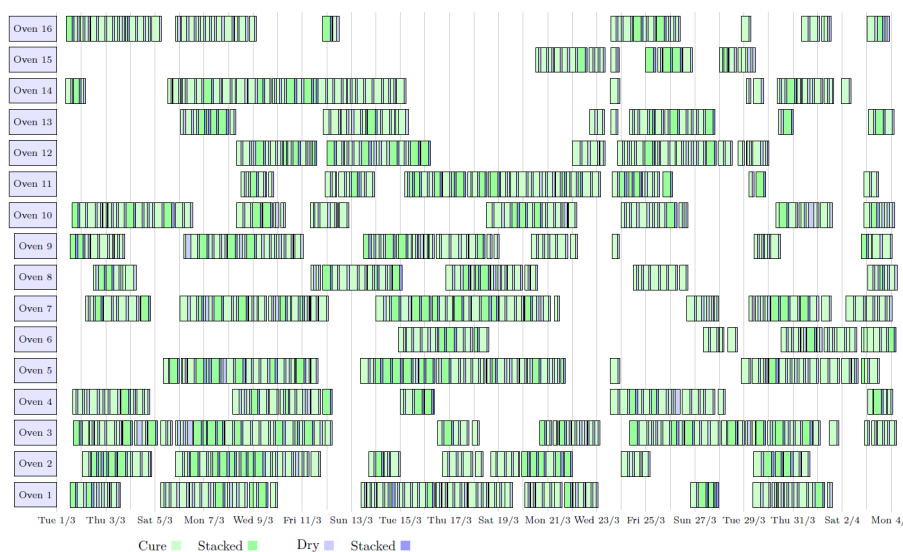


Are the short-term solutions good?

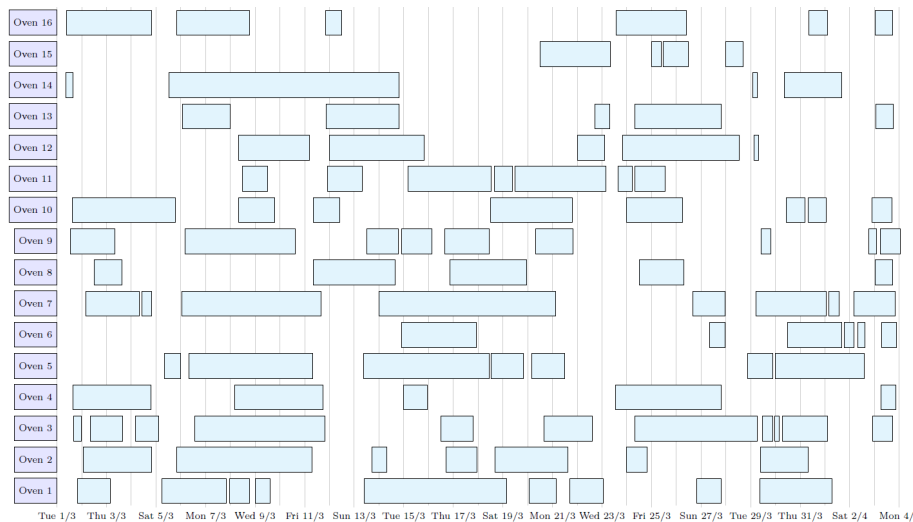


- We solve many problems to optimality, depending on solver
- Optimality gap is small, increasing search time helps a bit
- But are we optimizing the best possible objective?

Long Term Schedule: Detailed Schedule



Long Term Schedule: Abstracted Oven Runs



ENTIRE EDIH

Scheduling (One Day Course)

Slide 210

Is that a good global schedule? KPIs



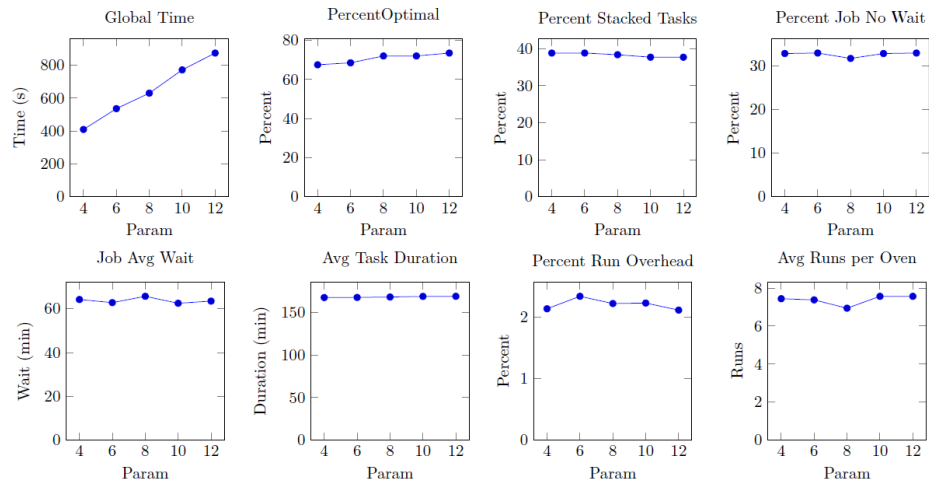
Name	Unit	Explanation
Global Time	Seconds	Total time for solving all sub problems
Nr Jobs	-	Total number of jobs scheduled
Nr Tasks	-	Total number of tasks scheduled
Percent Optimal	Percentage (0-100)	How many sub problems were solved to optimality
Percent Stacked Tasks	Percentage (0-100)	Percentage of all tasks scheduled that were stacked
Percent Jobs No Wait	Percentage (0-100)	Percentage of jobs that were scheduled without any waiting time
Job Average Wait	Minutes	Average wait time over all jobs
Job Maximal Wait	Minutes	Largest waiting time for any job scheduled
Ovens Used	-	Total number of ovens used during period
Avg Task Duration	Minutes	Average tasks duration (influenced by stacking)
Oven Runs	-	Number of oven runs over total horizon
Run Overhead Percent	Percentage (0-100)	Overhead during oven runs when machine is idle
Avg Runs per Oven Used	-	Average number of oven runs per oven used

ENTIRE EDIH

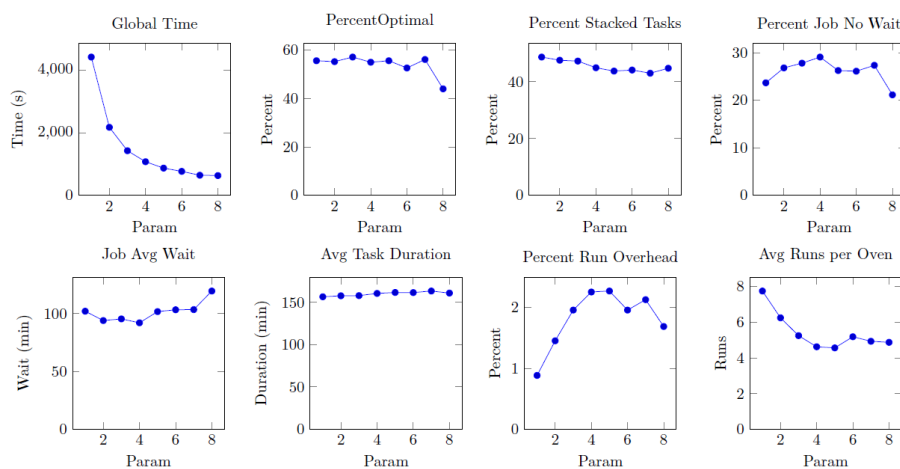
Scheduling (One Day Course)

Slide 211

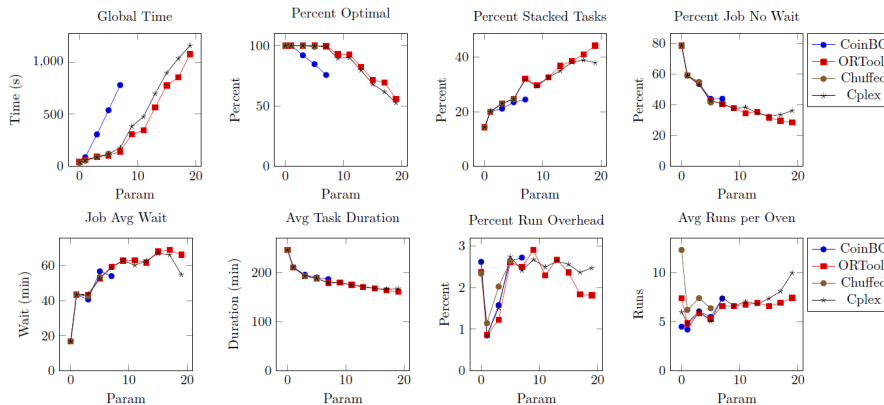
Impact of Lookahead Parameter



Impact of CommitHorizon Parameter



Comparing Different Solvers



Is the global solution really good?



- We schedule with limited information
- Hindsight is 20/20, we cannot expect best possible solution from partial information
- Process Challenge: Can we improve data visibility?
- Demand is variable over time, no steady-state solution
- Modelling Challenge: Can we define a short-term objective that produces better long-term solutions?
- Algorithm Challenge: Can we solve the global problem to optimality?
 - Assumes "a priori" visibility of data
 - This would provide a lower bound
 - But we need optimality to use as bound



- Discussed a non-standard oven scheduling problem from industry
- Models with decomposition of resource constraints
- Good/very good short-term solutions
- But is the overall schedule close to the global optimum?
- In any case, industry partner was happy with solution and analysis



Part VII

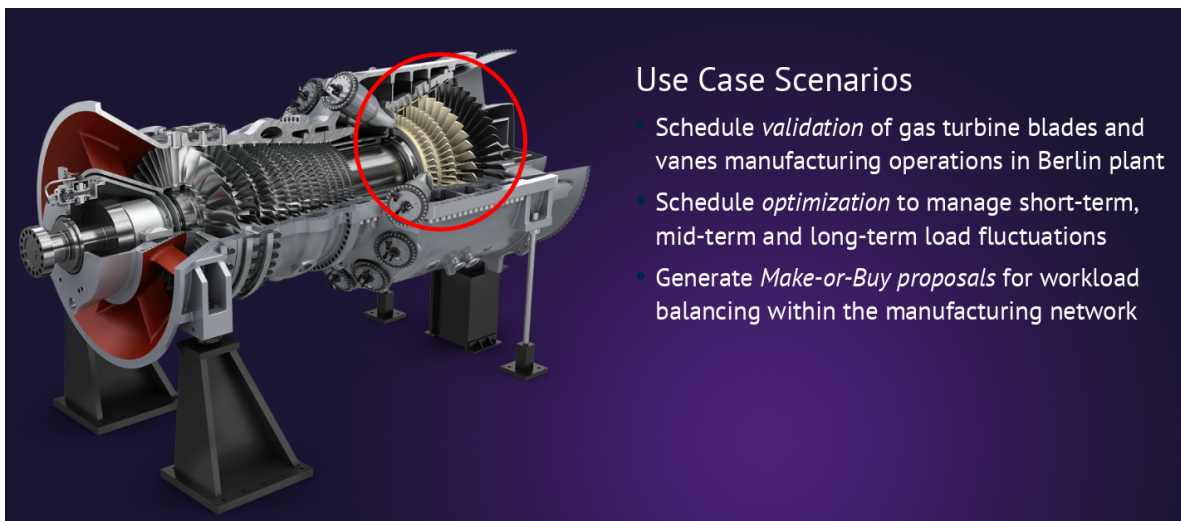
Blades and Vanes Production Case Study

Key Points



- Scheduling/Planning tool for manufacturing industry
- Developed as part of European ASSISTANT project
- Focused on key make-or-buy decisions
- Complex manufacturing process with alternative process paths
- Outperforms both current in-house tool and commercial simulator
- Key Technology: Optimization and Constraint Programming

Assistant Siemens Energy Use Case



Use Case Scenarios

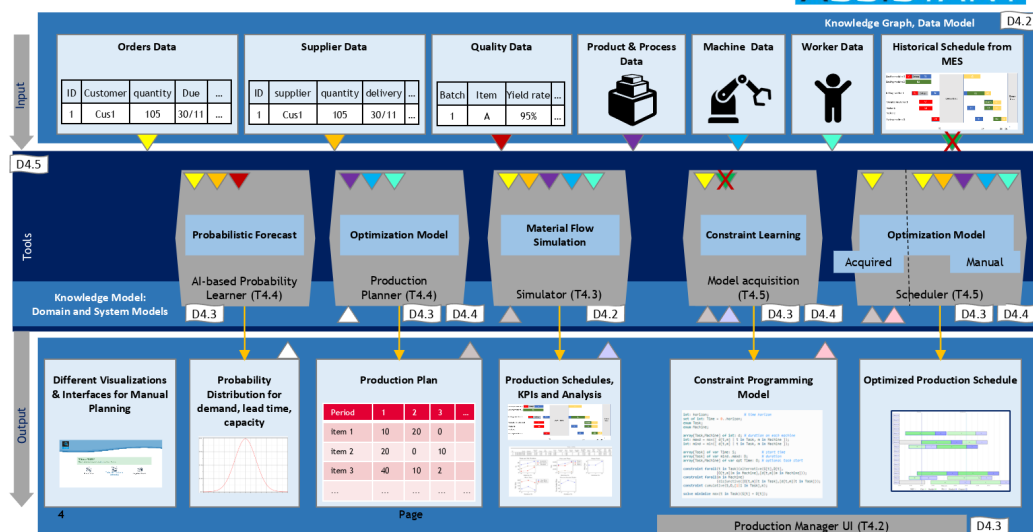
- Schedule *validation* of gas turbine blades and vanes manufacturing operations in Berlin plant
- Schedule *optimization* to manage short-term, mid-term and long-term load fluctuations
- Generate *Make-or-Buy proposals* for workload balancing within the manufacturing network

ASSISTANT Project Overview



Intelligent digital twin for process planning and scheduling

ASSISTANT

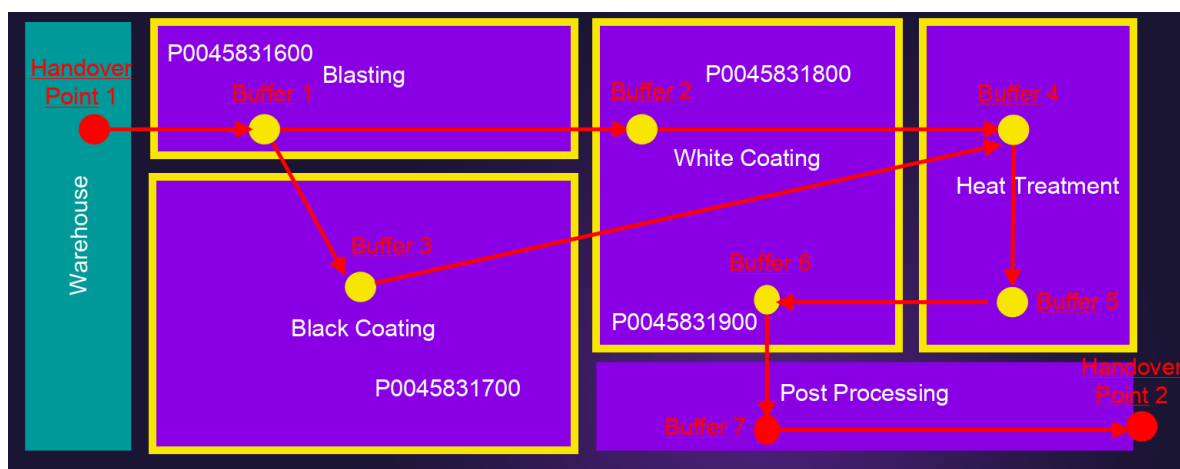


ENTIRE EDIH

Scheduling (One Day Course)

Slide 220

SE Product Routing



ENTIRE EDIH

Scheduling (One Day Course)

Slide 221

Full Scale Datasets

Berlin06: 96 orders, 9 months horizon, previous review

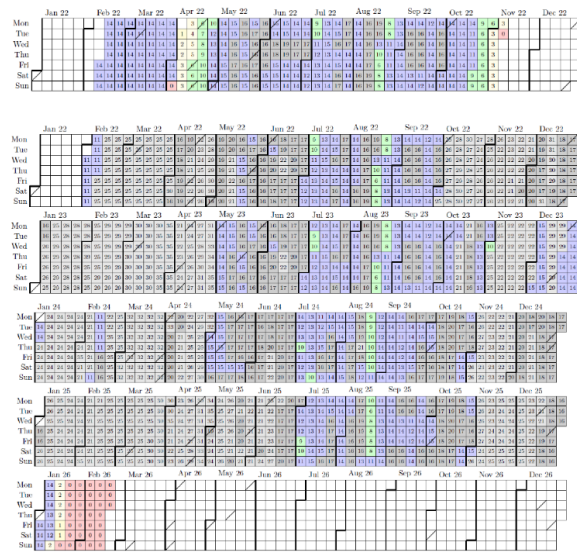
Berlin07: 450 orders, 4 years horizon

Berlin08: 559 orders, Christmas gap added

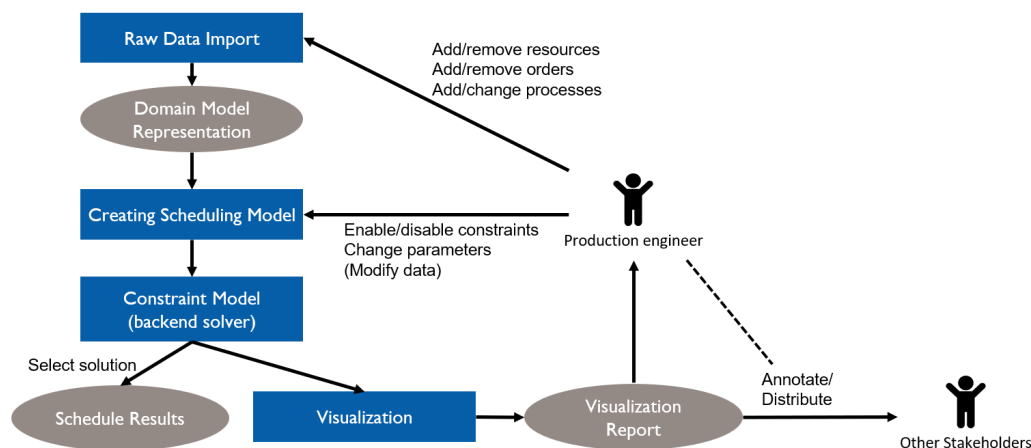
Berlin08a: 670 orders, filling gaps



Value in cell indicates active orders
Yellow and red colors indicate low order volume



Optimizer High Level Structure



Raw Data - Manual Data Entry Causes Problems



- Raw data come from spreadsheet
 - 20 tabs
- Excel is a particularly bad input data format
- Realistic, not real data
- Created by hand/automatically from existing test scenarios
- Series of files Berlin01 - Berlin05 were too inconsistent to run
- Berlin06 still contains some errors
- Optimizer explains all issues that it finds

ASSTANT Project Siemens Energy Use Case - Insight SFI Centre for Data Analytics

File Edit Scenario View Window Help

Raw Issue X

Name	Severity	Sheet	RowNo	ColNo	Description
Issue1	Major	t_Load	129	11	DateTime not formatted correctly, found 2022-02-28000000 format yyyy-MM-ddTTHH:mm:ss
Issue2	Minor	t_Products	1	15	Extra Empty Header
Issue3	Minor	t_Availability	1	8	Extra Empty Header
Issue4	Minor	t_Unavailabilities	1	8	Extra Empty Header
Issue5	Minor	t_Shift_Segments	1	6	Extra Empty Header
Issue6	Major	t_Shift_Segments	1	1	TimeOnly not formatted correctly, found 0.250000, format HH:mm:ss
Issue7	Major	t_Shift_Segments	1	2	TimeOnly not formatted correctly, found 0.550332, format HH:mm:ss
Issue8	Major	t_Shift_Segments	2	1	TimeOnly not formatted correctly, found 0.291667, format HH:mm:ss
Issue9	Major	t_Shift_Segments	2	2	TimeOnly not formatted correctly, found 0.300083, format HH:mm:ss
Issue10	Major	t_Shift_Segments	3	1	TimeOnly not formatted correctly, found 0.458333, format HH:mm:ss
Issue11	Major	t_Shift_Segments	3	2	TimeOnly not formatted correctly, found 0.479167, format HH:mm:ss
Issue12	Major	t_Shift_Segments	4	1	TimeOnly not formatted correctly, found 0.563333, format HH:mm:ss
Issue13	Major	t_Shift_Segments	4	2	TimeOnly not formatted correctly, found 0.816667, format HH:mm:ss
Issue14	Major	t_Shift_Segments	5	1	TimeOnly not formatted correctly, found 0.866667, format HH:mm:ss
Issue15	Major	t_Shift_Segments	5	2	TimeOnly not formatted correctly, found 0.877083, format HH:mm:ss
Issue16	Major	t_Shift_Segments	6	1	TimeOnly not formatted correctly, found 0.770833, format HH:mm:ss
Issue17	Major	t_Shift_Segments	6	2	TimeOnly not formatted correctly, found 0.791667, format HH:mm:ss
Issue18	Major	t_Shift_Segments	7	1	TimeOnly not formatted correctly, found 0.916667, format HH:mm:ss
Issue19	Major	t_Shift_Segments	7	2	TimeOnly not formatted correctly, found 0.250000, format HH:mm:ss
Issue20	Major	t_Shift_Segments	8	1	TimeOnly not formatted correctly, found 0.000000, format HH:mm:ss
Issue21	Major	t_Shift_Segments	8	2	TimeOnly not formatted correctly, found 0.010417, format HH:mm:ss
Issue22	Major	t_Shift_Segments	9	1	TimeOnly not formatted correctly, found 0.083333, format HH:mm:ss
Issue23	Major	t_Shift_Segments	9	2	TimeOnly not formatted correctly, found 0.104167, format HH:mm:ss
Issue24	Minor	t_Shift_Segments	10	0	First Column Empty
Issue25	Minor	t_Shift_Segments	11	0	First Column Empty
Issue26	Minor	t_Shift_Segments	12	0	First Column Empty
Issue27	Minor	t_Shift_Segments	13	0	First Column Empty
Issue28	Minor	t_Shift_Segments	14	0	First Column Empty
Issue29	Minor	t_Shift_Segments	15	0	First Column Empty
Issue30	Minor	t_Shift_Segments	16	0	First Column Empty
Issue31	Minor	t_Shift_Segments	17	0	First Column Empty
Issue32	Minor	t_Shift_Segments	18	0	First Column Empty
Issue33	Minor	t_Shift_Patterns	1	9	Extra Empty Header
Issue34	Minor	t_Shift_Patterns	7	0	First Column Empty
Issue35	Minor	t_Shift_Patterns	8	0	First Column Empty

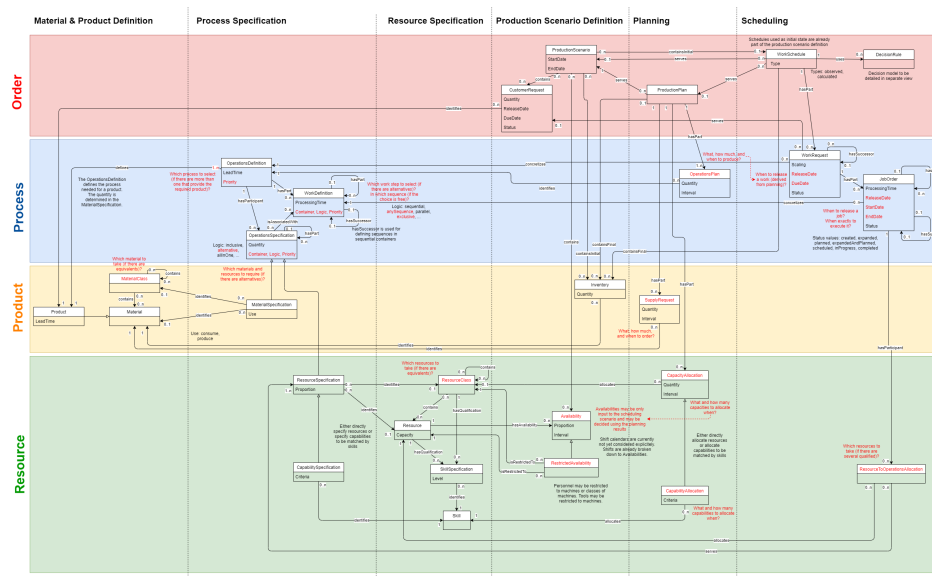
Filter

ENTIRE EDIH

Scheduling (One Day Course)

Slide 224

Domain Model - Knowledge Graph

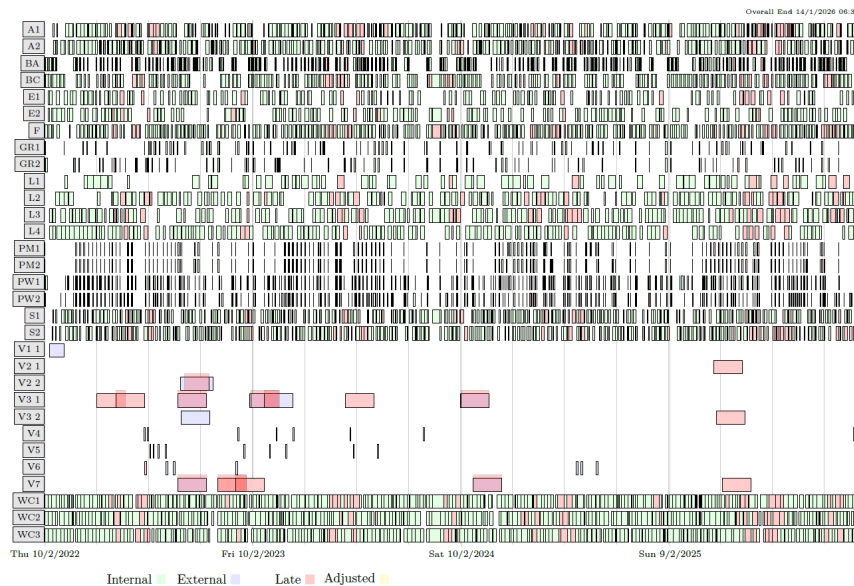


ENTIRE EDIH

Scheduling (One Day Course)

Slide 225

Solution for Berlin 08a - Shows Only 20% of Tasks in Model



Implementation

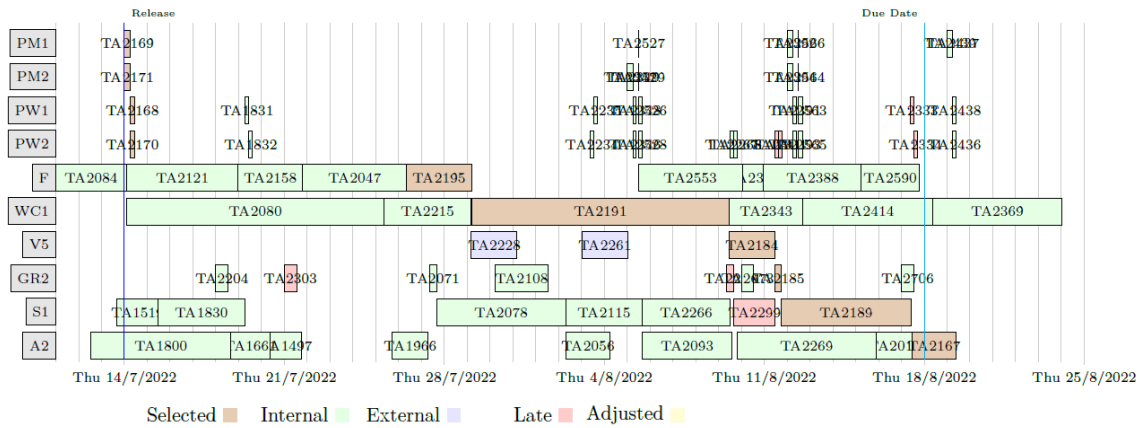


- Requirement capture done inside project
- Data checking/cleaning most time consuming aspect
- Some specified functionality was rejected by Betriebsrat
- Built in Java
- Uses IBM's CPOptimizer back-end
- 120k LoC, 110k generated, 3k solver
- Outperforms both
 - Current in-house tool
 - Simulation based tool based on commercial simulator
- System installed at SE site, but not in daily use

Explaining Late Delivery



- Explain why some orders are delivered late
- Find root-cause, show schedule in context



Evaluation - KPIs



KPI	Baseline	Optimizer
OTD	> 80 %	92 %
Bottleneck machine utilization	99.5 %	100 %
Manufacturing defects	10-15 %	< 10 %
Scenarios in 8 hours	15-20	> 100,000



*“Within less than eight hours the **ASSISTANT** tools provided us thousands of manufacturing scenarios including different make-or-buy recommendations for making deliberate decisions on the way to proceed for strategic planning.”*

from ASSISTANT final project review: Siemens Energy assessment

Summary



- Scheduling/Planning tool for manufacturing industry
- Developed as part of European ASSISTANT project
- Focused on key make-or-buy decisions
- Complex manufacturing process with alternative process paths
- Outperforms both current in-house tool and commercial simulator
- Key Technology: Optimization and Constraint Programming

Part VIII

Experiments

Key Points



- This section describes our scheduling tool
 - This is a *demonstrator*, not an industrial product!
- How to load/create data
 - From files
 - By instance generator
 - From benchmark problems
- How to run the solvers
 - Which solvers are supported
 - What to expect in terms of performance
- Experiments to try
 - Limited time
 - Possible "test before invest" continuation

The Scheduling Tool



- We create the tool as basis for experiments
- To test ideas and solvers
- As a teaching tool
- Slightly higher standard than usual academic prototypes
 - This is a *demonstrator*
- Not a commercial tool
 - But can use commercial solvers
 - Also open-source solvers
- Written in Java, JavaFX
- Can also be used as a back-end scheduling server
- Uses our Java application framework generator
- Sources available on request

Back-end solvers



- Provide both open-source and commercial solver interfaces
- Allow experimentation without having to buy commercial tools straightaway
- Gives a level playing field to compare solvers and models
- Provides out-of-the-box, generic performance

Google OR-Tools CPSat Solver



- Open-Source tool provided by Google
- Available at https://developers.google.com/optimization/cp/cp_solver
- Probably best open-source CP solver for scheduling
- This solver is packaged with scheduler

Example Problem

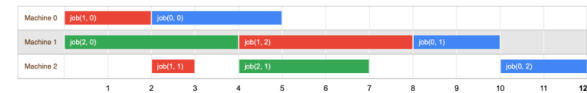
Below is a simple example of a job shop problem, in which each task is labeled by a pair of numbers (m, p) where m is the number of the machine the task must be processed on and p is the *processing time* of the task – the amount of time it requires. (The numbering of jobs and machines starts at 0.)

- Job 0 = $[(0, 3), (1, 2), (2, 2)]$
- Job 1 = $[(0, 2), (2, 1), (1, 4)]$
- Job 2 = $[(1, 4), (2, 3)]$

In the example, Job 0 has three tasks. The first, $(0, 3)$, must be processed on machine 0 in 3 units of time. The second, $(1, 2)$, must be processed on machine 1 in 2 units of time, and so on. Altogether, there are eight tasks.

A solution for the problem

A solution to the job shop problem is an assignment of a start time for each task, which meets the constraints given above. The diagram below shows one possible solution for the problem:



You can check that the tasks for each job are scheduled at non-overlapping time intervals, in the order given by the problem.

The length of this solution is 12, which is the first time when all three jobs are complete. However, as you will see [below](#), this is not the optimal solution to the problem.

(from OR-Tools website)

ENTIRE EDIH

Scheduling (One Day Course)

Slide 238

CP Optimizer from IBM



- Commercial tool of IBM
- <https://www.ibm.com/products/ilog-cplex-optimization-cplex-cp-optimizer>
- Part of optimization suite with Cplex, OPL
- We do **not** provide this solver, we allow to interface with it
- Academic licenses available
- Well-known for capabilities for scheduling

Resources



Applications of constraint programming

Explore applications of constraint programming including production problem and scheduling use cases.

[Read the documentation](#) →

(from CPOptimizer website)

ENTIRE EDIH

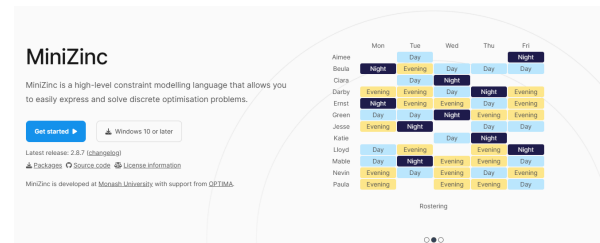
Scheduling (One Day Course)

Slide 239

MiniZinc from Monash University



- Modelling language and backend tools from Monash University in Melbourne, Australia
- Available from <https://www.minizinc.org/>
- Widely used for teaching
- Allows different backend solver to run from same model
- Generic CP tool, not optimized for scheduling
- Requires separate installation, open-source



(from MiniZinc Website)

Which Solver is Better?



- We present results on a few benchmark types
- Fair comparison between solvers
 - Same hardware, Windows 11 laptop
 - CPU i7-10875H @ 2.3GHz, 64GB, four cores
 - Same timeout (600 s)
- Not a fair comparison to state-of-the-art
 - Uses out-of-the-box model
 - Significant improvements possible
 - More specific models
 - Parameter tuning
 - Unlimited runtime

Group	Nr	All Instances				Optimal Only		Non Optimal Only			
		Optimal (% of All Instances)				Time (% of VB)		Cost (% of VB)		Bound (% of VB)	
		Both	CPO	CPSat	None	CPO	CPSat	CPO	CPSat	CPO	CPSat
15/15	10	90.00	0.00	0.00	10.00	105.19	141.18	100.00	100.00	97.17	100.00
20/15	10	20.00	0.00	0.00	80.00	267.27	263.20	100.99	100.05	98.50	99.93
20/20	10	0.00	0.00	0.00	100.00	n/a	n/a	100.74	100.06	97.96	100.00
30/15	10	10.00	0.00	10.00	80.00	174.32	100.00	100.18	100.49	99.87	100.00
30/20	10	0.00	0.00	0.00	100.00	n/a	n/a	100.30	101.30	99.40	100.00
50/15	10	100.00	0.00	0.00	0.00	100.00	685.09	n/a	n/a	n/a	n/a
50/20	10	10.00	60.00	0.00	30.00	100.00	381.38	100.00	101.60	100.00	100.00
100/20	10	10.00	90.00	0.00	0.00	100.00	416.13	100.00	101.73	100.00	66.81

- Significant number of problems solved to optimality in 600s
- In terms of quality, solvers are quite similar
- CPO wins in terms of solution times for larger instances

Results for Hybrid Flexible Flow-Shop

Group	Nr	All Instances				Optimal Only		Non Optimal Only			
		Optimal (% of All Instances)				Time (% of VB)		Cost (% of VB)		Bound (% of VB)	
		Both	CPO	CPSat	None	CPO	CPSat	CPO	CPSat	CPO	CPSat
20	25	76.00	0.00	20.00	4.00	100.00	580.71	100.00	100.00	96.52	100.00
25	25	80.00	0.00	8.00	12.00	101.65	238.02	100.00	100.37	97.67	100.00
30	25	60.00	0.00	4.00	36.00	100.35	264.69	100.18	101.05	100.00	100.00
40	25	4.00	16.00	0.00	80.00	100.00	2554.03	100.00	104.68	100.00	100.00
50	25	0.00	4.00	0.00	96.00	n/a	n/a	100.00	107.87	100.00	100.00
100	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	120.43	100.00	100.00
200	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	188.60	100.00	100.00
300	24	0.00	0.00	0.00	100.00	n/a	n/a	100.00	263.22	100.00	100.00
400	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	246.34	100.00	100.00

- Only smaller/medium instances solved to optimality
- For those problems, both solvers perform well
- CPO significantly better on large instances



- If you already have access to CPO, use it!
- For new problem types, do an evaluation with CPSat first
- Out of the box, CPO performs more consistently
- May be easier to extend CPSat with your own research
- Use multiple cores and memory to your advantage

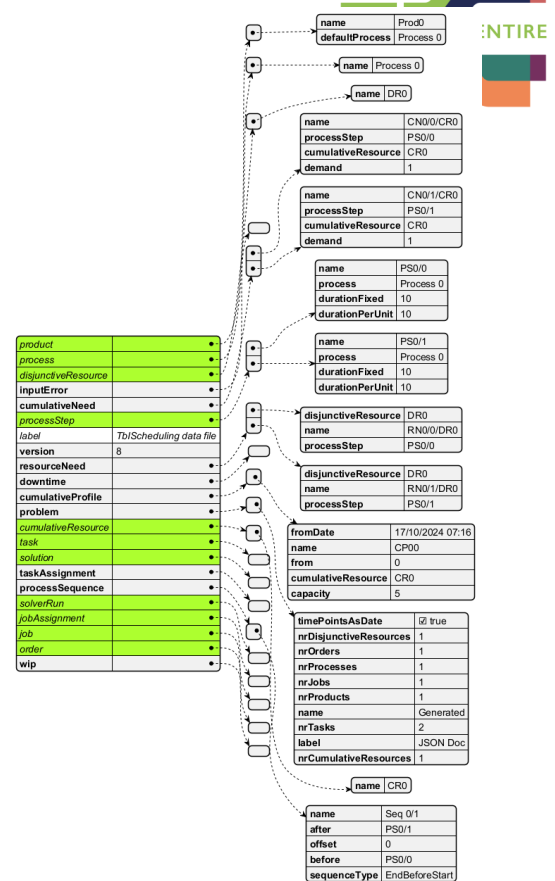
Input Data



- We have defined a specific JSON data format to describe scheduling problems
- This is different from the native/XML data format of the application (do not use)
- Load with menu `File - Load DataFile...`
- Save with menu `File - Save DataFile...`
- The format is described in a document

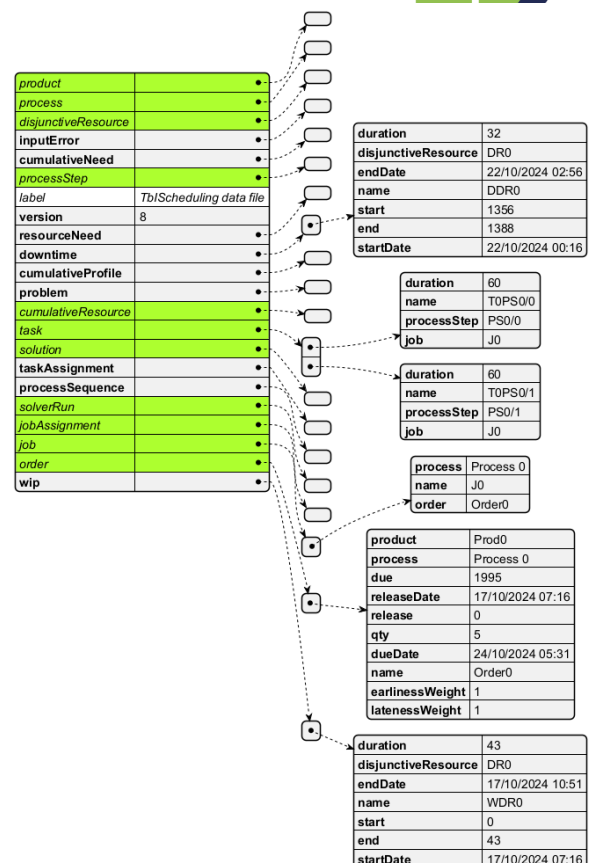
Base Data

- Description of
 - Product
 - Process
 - DisjunctiveResource
 - CumulativeNeed
 - ProcessStep
 - ResourceNeed
 - CumulativeProfile
 - Problem
 - CumulativeResource
 - ProcessSequence



Schedule Input Data

- Description of
 - Downtime
 - Task (x2)
 - Job
 - Order
 - WiP



- We use the same JSON format to describe the results of the schedule
- Added field types for SolverRun, Solution, assigned Jobs and Tasks

Sample Results

- Description of
 - Solution
 - SolverRun
 - Job Assignment
 - Task Assignment



product	103
process	1032
disjunctiveResource	17/10/2024 20:51
inputError	0
totalWaitBefore	0
totalEarly	1
percentEarly	100
duration	120
maxLateness	0
maxWaitAfter	0
inLate	0
gfp	0
solverRun	Run1
end	103
weightedEarliness	1032
totalLateness	0
maxWaitBefore	0
bound	103
solverStatus	Optimal
start	43
totalWaitAfter	0
percentLate	0
weightedLateness	0
maxEarliness	1032
name	So11
objectiveValue	103
flowtime	103
startDate	17/10/2024 10:51

duration	60
waitAfter	0
task	T0P500
disjunctiveResource	DR0
endDate	17/10/2024 15:51
name	TA0
start	43
end	103
jobAssignment	JA0
waitBefore	0
startDate	17/10/2024 10:51

duration	60
waitAfter	0
task	T0P501
disjunctiveResource	DR0
endDate	17/10/2024 20:51
name	TA1
start	103
end	103
jobAssignment	JA0
waitBefore	0
startDate	17/10/2024 15:51

nThreads	2
weightEarliness	1
seed	42
solverBackend	None
weightMakespan	1
producePDF	☐ false
solverStatus	Optimal
description	
label	
modelType	CPO
produceWp	☒ true
objectiveType	Makespan
timeout	30
weightFlowtime	1
enforceCumulative	☒ true
name	Run1
weightLateness	1
time	0.142
produceReport	☐ false
enforceOverdime	☒ true
removeSolution	☐ false
enforceReleaseDate	☒ true
enforceDueDate	☐ false

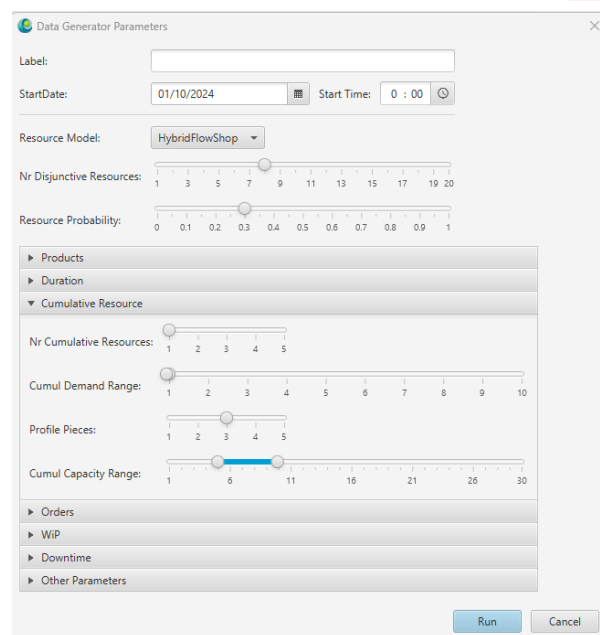
duration	100
solution	So11
late	0
endDate	17/10/2024 20:51
name	JA0
start	43
end	103
job	JA
early	1032
startDate	17/10/2024 10:51

- Application allows to generate different types of test problems
- Different types of resource models
- Different numbers of orders, resources, WiP, downtime
- Useful to generate more life-like examples combining different constraint types

Instance Generator Dialog



- Resource Model
 - Select a resource model defining the overall structure of problem
- Nr Disjunctive Resources
 - Describe how many disjunctive resources are generated
- Resource Probability
 - The probability that a resource is compatible with a task
 - Only for some resource models



The screenshot shows the 'Data Generator Parameters' dialog box. It includes fields for 'Label', 'StartDate' (01/10/2024), and 'Start Time' (0 : 00). The 'Resource Model' is set to 'HybridFlowShop'. Below this are sliders for 'Nr Disjunctive Resources' (range 1 to 20, value at 7), 'Resource Probability' (range 0 to 1, value at 0.3), 'Nr Cumulative Resources' (range 1 to 5, value at 1), 'Cumul Demand Range' (range 1 to 10, value at 1), 'Profile Pieces' (range 1 to 5, value at 3), and 'Cumul Capacity Range' (range 1 to 30, value at 6). There are expandable sections for 'Products', 'Duration', 'Cumulative Resource', 'Orders', 'WiP', 'Downtime', and 'Other Parameters'. 'Run' and 'Cancel' buttons are at the bottom right.

Resource Models



- Flow-Shop
 - Multiple stages, all jobs use machines in same order
- Job-Shop
 - Multiple stages, jobs use machines in different order
- Open-Shop
 - Multiple stages, no predefined order of machines
- Hybrid Flow-Shop (default)
- Hybrid Job-Shop
- Hybrid Open-Shop
 - Like x-shop, but with multiple machines per stage
- Random
 - Multiple stages, each stage using a random subset of machines
- All
 - Multiple stages, each stage allowing all machines

Instance Generator - Products



- Nr Products
 - Number of products to be generated
 - Products may be reused by multiple orders
- Stages Range
 - Range slider, sets lower and upper bound on number of stages

A screenshot of the "Data Generator Parameters" dialog box. It contains several input fields and sliders. The "Label" field is empty. The "StartDate" is set to "01/10/2024" and the "Start Time" is "0 : 00". The "Resource Model" is set to "HybridFlowShop". The "Nr Disjunctive Resources" slider is set to 7. The "Resource Probability" slider is set to 0.3. The "Products" section is expanded, showing "Nr Products" set to 10 and the "Stages Range" slider set to 4. Other sections like "Duration", "Cumulative Resource", "Orders", "WIP", "Downtime", and "Other Parameters" are collapsed. "Run" and "Cancel" buttons are at the bottom right.

Instance Generator - Duration



- Duration Model
 - Different ways to link duration of processSteps
- Duration Range
 - Range slider to set lower and upper bounds on perUnit duration
- Duration Fixed Factor
 - How fixed and perUnit duration values are linked

Instance Generator - Cumulative



- Nr Cumulative Resources
 - Number of cumulative resources generated
- Cumul Demand Range
 - Range slider to select lower and upper bound on cumulativeResource-Need demands
- Profile Pieces
 - Number of segments of CumulativeProfile generated for each resource
- Cumul Capacity Range
 - Range slider to select lower and upper bounds on cumulative profile capacity values

Instance Generator - Orders



- Nr Orders
 - Number of orders generated, each order is assigned a random product/process
- Qty Range
 - Range slider to select lower and upper bounds on quantity for each order

The screenshot shows the 'Data Generator Parameters' dialog box with the 'Orders' section expanded. The 'Nr Orders' is set to 50. The 'Qty Range' slider is set from 1 to 11. Other sections like 'Products', 'Duration', 'Cumulative Resource', 'WIP', 'Downtime', and 'Other Parameters' are collapsed. The 'Run' and 'Cancel' buttons are at the bottom right.

Instance Generator - WiP (Work in Progress)



- WiP Probability
 - Probability of generating a WiP for a disjunctive resource
- WiP Range
 - Range slider to set lower and upper bound on WiP duration

The screenshot shows the 'Data Generator Parameters' dialog box with the 'WiP' section expanded. The 'WiP Probability' slider is set to 0.5. The 'WiP Range' slider is set from 21 to 51. Other sections like 'Products', 'Duration', 'Cumulative Resource', 'Orders', 'Downtime', and 'Other Parameters' are collapsed. The 'Run' and 'Cancel' buttons are at the bottom right.

Instance Generator - Downtime



- Downtime Probability
 - Probability of generating a downtime for a disjunctive resource
- Downtime Range
 - Range slider to select lower and upper bounds on downtime duration

The screenshot shows the 'Data Generator Parameters' dialog box with the 'Downtime' section expanded. The 'Downtime Probability' slider is set to 0.5, and the 'Downtime Range' slider is set from 51 to 71. The 'Run' button is highlighted.

Instance Generator - Other Parameters



- Earliest Due
 - Smallest allowed value for a due date
- Horizon Days
 - What planning horizon to consider (in days)
- Time Resolution
 - In minutes, links internal and external time presentation
- Random Seed
 - Random seed to make reproducible random choices

The screenshot shows the 'Data Generator Parameters' dialog box with the 'Other Parameters' section expanded. The 'Earliest Due' field is set to 20, 'Horizon Days' is set to 20, 'Time Resolution' is set to 5, and 'Random Seed' is set to 42. The 'Run' button is highlighted.



- Three datasets of different sizes
 - Job-shop
 - Flow-shop
 - Open-shop
- Load with menu `File - Load DataFile...` -
`Taillard -`
- Larger instances need more solver time to reach good solutions (600 s)

Simple Assembly Line Balancing Problem (SALBP)



- Will be discussed on more details as case study
- Design an assembly line setup by solving a scheduling problem
- Balance a set of operations across a number of stations of an assembly line
- Precedence graph is not a chain, can be very complex
- Specialized problem normally solved with specialized tools
- Load with menu `File - Load SALBP Problem...`



- Will be discussed in more details as case study
- Schedule a set of tests on a number of machines, minimizing total duration
- Single stage tests, possibly large number of resources
- Closely related to bin-packing
- Load with menu `File - Load Test Scheduling Problem...`

Experiment 1



- Start the application
 - Our running example will be automatically generated
- Look at the process diagram `Window-Product-Process Diagram`
- Run the solver `Scenario - Run ScheduleJobs Solver`
- Observe the results in Gantt Chart
- Customize display
- Look at Cumulative Resource Chart
`Window-Solution-Cumulative Resource Chart`

Experiment 2



- Re-run solver disabling cumulative constraint
- Observe result in Gantt chart
- See impact on Cumulative Resource chart
- Switch between solutions in charts

Experiment 3



- Check Gantt chart display for delayed tasks, enabling lateness display
- Re-run solver, enforcing due-date constraints
- What impact does this have on objective

Experiment 4



- Change objective to on-time delivery
- Results are very different, why?
- More explanations on this tomorrow

Experiment 5



- Load one of the other example types
- For example, Taillard Job-shop 15x15
- Understand process diagram
- Run solver
- Look at intermediate solutions found

Summary



- We presented an overview of our generic scheduling tool
- Discussed available solvers, both commercial and open-source
- Described the JSON data format for input and output
- Gave an overview of the instance generator provided
- Shows example problems included with tool
- Suggested some experiments to run

Survey



- Please fill in before you leave!

